

# ספר פרויקט

שם המכללה: בסמח

שם מגמת הלימוד: הנדסת תוכנה

מסלול הכשרה: טכנאים מוסמכים

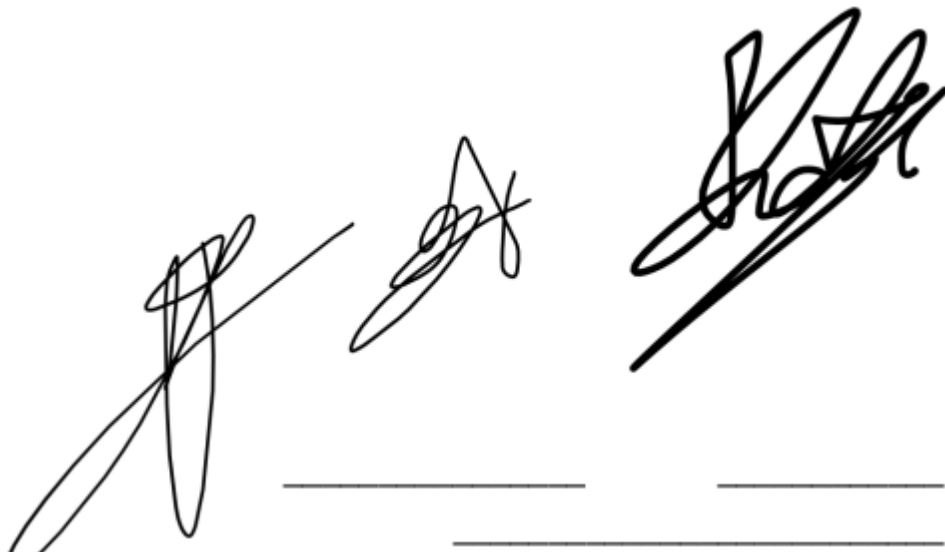
נושא פרויקט הגמר: Bird Radar – אתר הצפרות הלאומי

שם המגישים: עדן סבח ויקטרינה יבטוחובסקי

שם המנחה האישי: עילי גילת

תאריך הגשת ספר הפרויקט: 11.03.2025

חתימת הסטודנט:



## תוכן עניינים

|    |                              |
|----|------------------------------|
| 1  | פתיח.....                    |
| 3  | הצעת פרוייקט.....            |
| 19 | הצהרת הסטודנטים.....         |
| 22 | הוקרות.....                  |
| 23 | תקציר.....                   |
| 24 | עקרונות בסיסיים בפרויקט..... |
| 25 | תרשים אי אר די.....          |
| 26 | צד שרת.....                  |
| 39 | צד לקוח.....                 |
| 67 | תהליכים מרכזיים במערכת.....  |
| 71 | סיכום ומסקנות.....           |
| 72 | ביבליוגרפיה.....             |

חוזר מנהל מה"ט 51-4-11 – נספח מס 1' (הצעה לפרויקט גמר)

תאריך: 29/09/2024

לכבוד  
יחידת הפרויקטים  
מה"ט

הצעה לפרויקט גמר

1. פרטי הסטודנטים

| שם הסטודנט | תעודת זהות - 9 ספרות | כתובת | טלפון נייד | תאריך סיום הלימודים |
|------------|----------------------|-------|------------|---------------------|
|------------|----------------------|-------|------------|---------------------|

|   |            |                              |           |                       |
|---|------------|------------------------------|-----------|-----------------------|
| - | 0585122139 | אבן יהודה<br>העצמאות 58      | 325879666 | עדן סבח               |
| - | 0542101968 | ראשון לציון מעגל<br>השלום 23 | 326654225 | יקטרינה<br>יבטוחובסקי |

שם המכללה: בסמח.

סמל המכללה: 71605.

מסלול ההכשרה: טכנאים מוסמכים.

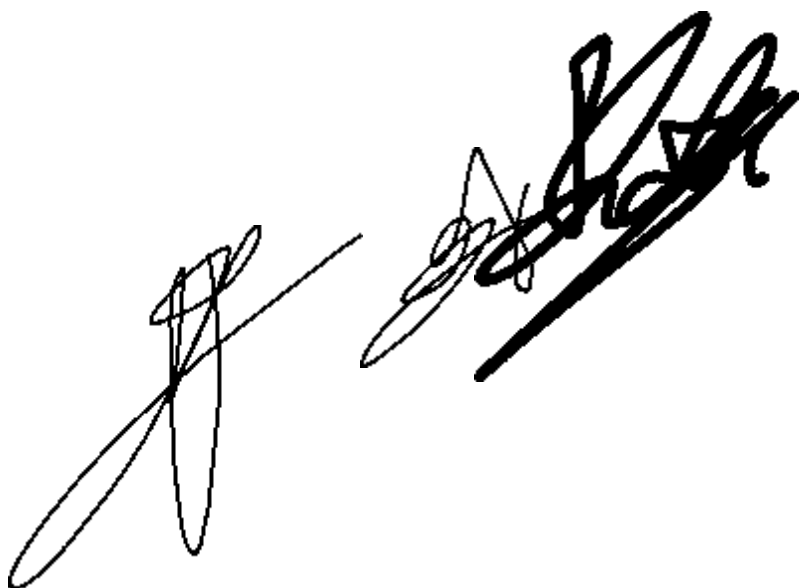
מגמת לימוד: הנדסת תוכנה.

מקום ביצוע הפרויקט: בסמח.

## 2. פרטי המנחה האישי

| שם המנחה  | כתובת                     | טלפון נייד | תואר                      | מקום<br>עבודה/תפקיד |
|-----------|---------------------------|------------|---------------------------|---------------------|
| עילי גילת | האירוסים 67<br>קדימה צורן | 0526010916 | מדעי המחשב<br>מכללת נתניה | צהל                 |

\* דיפלומת המנחה מצורפת בסוף המסמך



## **1. שם הפרויקט - Bird Radar**

### **2. רקע**

#### **2.1. תיאור ורקע כללי:**

מרכז הצפרות הישראלי פועל למען שמירה על מגוון מיני העופות בישראל, שימור ושיקום בתי הגידול החשובים לקיומם. אנו שואפים לחזק את התמיכה הציבורית בשמירת הטבע בישראל על ידי שיתוף פעולה של הציבור ביחד עם המדינה על מנת לשמור על כל מיני הציפורים בישראל. מדי שנה, אלפי זנים של ציפורים מבקרים את ארצנו היפה, ועדיין אין דרך לדווח היכן הן נמצאות, לאן פניהן מועדות ולגלות היכן נמצאים הזנים הנדירים.

#### **2.2. מטרות המערכת:**

המערכת לניהול כל זני הציפורים בישראל, בה אפשר לדווח על נוכחות ציפורים במקום מסוים, כמו גם להוסיף פירוט במידה ומדובר בזן נדיר, וכך הציבור יגלה יותר ערנות כלפי הטבע בישראל ויתחבר יותר לעולם הצפרות שהולך ודועך עם השנים.

## **3. סקירת מצב קיים בשוק, אילו בעיות קיימות?**

בזבוז זמן: צפרים כיום בישראל משקיעים זמן רב בחיפוש ובתצפיות אחר זני הציפורים עליהם הם חוקרים באותו זמן, וקשה להעביר את המידע בצורה נוחה בין קהילת הצפרים בישראל.

היעדר זיקה לטבע בישראל: על אף עשרות קמפיינים הציבור עדיין ממשיך לזהם ולפגוע בטבע הישראלי, בחוסר מודעות שהדבר פוגע בציפורים החיות בסביבתנו.

היכחדות הציפורים: הציפורים אט אט נכחדות, ובעוד מספר שנים לא גבוה במיוחד, זני הציפורים הנדירים יעלמו ואנחנו לא נזכה לתיעוד שלהם והזנים האלה ישארו בדפי ההיסטוריה.

#### 4. מה הפרויקט אמור לחדש או לשפר?

ייעול תהליכים: המערכת מאפשרת איחוד של מידע ממגוון מקורות שונים, מצפרים מכל הארץ, כמו גם חובבי טבע ואפילו מהאדם הפשוט.

שיפור חווית המשתמש: יש ממשק משתמש נוח ופשוט לצפרים ולכלל הציבור הישראלי.

גיבוש לכל המשפחה: האתר יאפשר למשפחות לגלות את הטבע וימשוך את ילדיהם לחקור על סוגי ציפורים וללמוד עליהם.

#### 1. דרישות מערכת ופונקציונאליות

##### 4.1. דרישות מערכת:

המערכת תשמש את כלל הציבור הישראלי, החל בצפרים מקצועיים, ועד ילדים סקרנים המתעניינים בטבע. על המערכת לעדכן את המידע באופן מהיר ולשמור כמויות גדולות של מידע, שכן הציפורים נעות ממקום למקום ומיקומן משתנה במהירות. על המערכת לדעת לקחת מידע מהמון משתמשים ולהציג אותו על המפה באופן נקי ויעיל.

בנוסף, על המערכת שלנו להיות נגישה וידידותית לכלל המשתמשים. המערכת תאפשר ממשק ידידותי לילדים, כך שהיא פונה לכל הקהל הישראלי. המערכת תציג פירוט על סוג הציפור וכך התהליך ינוהל באופן מהיר ויחסוך חיפושים במקורות מידע אחרים.

##### 4.2. דרישות פונקציונאליות:

למערכת יהיה מסך של התחברות, הכולל הרשמה או התחברות ובחירת סוג המשתמש (צפר/חובב טבע/ילד)

-מסך הבית של המערכת - מפה ועליה הציפורים הנצפו על ידי משתמשי המערכת. זהו העמוד המרכזי במערכת. בצידה יהיה תפריט בו ניתן לבצע מגוון פעולות - סינון הציפורים לפי סוגים, מעבר למסך המידע על כל ציפור, דיווח על צפייה של ציפור.

-מסך המידע - מסך בו ניתן לצפות במידע על כל מיני סוגי ציפורים, ללמוד ולהחכים.

-מסך ילדים (היופיע רק במידה וסוג המשתמש הרשום הינו ילד) - מסך אינטרקטיבי עם מגוון משחקים הקשורים לעולם הצפרות.

- מסך הבקשות – מסך היופיע למשתמש רק במידה והינו צפר, הצפר יוכל לנהל את בקשות שאר המשתמשים לדיווח של ציפורים. במידה והוא יאשר את הבקשה היא תישמר ותופיע על המפה, במידה ולא המשתמש המדווח יותרע על כך והבקשה לא תישמר.

## 1. -בעיות צפויות במהלך הפיתוח ופתרונות (תפעוליות, טכנולוגיות, עומס ועוד):

### 6.1. בעיות:

עומסים על המערכת: למערכת מגוון משתמשים: כל אחד יכול להיכנס ולדווח על מידע בהפרשי זמני קטנים ולכן עליה להתמודד עם עומס.

כפילויות מידע: יכולים להימצא מספר משתמשים בקרבת מקום, והדבר יגרום על דיווח על אותו סוג ציפור מספר פעמים, לפעמים גם בחוסר תיאום.

### 6.2. פתרונות:

#### עומסים:

**Auto Scaling:** אנחנו משתמשים בKubernetes : פלטפורמה אשר בין היתר מכילה מנגנון אוטומטי לניהול עומסים. מאפשרת להוריד או להוסיף קונטיינרים באופן אוטומטי על פי העומס בנקודת הזמן הנוכחית. זה מאפשר לנו להגיב בזמן אמת לשינויים בעומס ולהבטיח שהשרתים יכולים לטפל במספר הבקשות הגבוה.

#### כפילויות מידע:

עבור כל הזנה של המשתמש, לפני פרסומה לשאר המשתמשים תתבצע בדיקה האם פורסמה הזנה דומה של אותו סוג ציפור ברדיוס קרוב ובזמן חופף. אם כן, יותרע למשתמש שמישהו אחר כבר פרסם תיעוד לציפור זו ואין צורך בתיעוד נוסף.

## 5. פתרון טכנולוגי נבחר:

5.1. מערכת וובית שתהיה מורכבת מצד לקוח client, צד שרת (server), dbi, המערכת תפרש באמצעות תהליך CI/CD לשרתים.

### טכנולוגיות בשימוש:

5.2. **Angular** - אחד הכלים הכי פופולריים בשוק לבניית צד לקוח במערכות ווביות, מערכות כדוגמת גוגל, ג'ימייל, פייפאל נטפליקס ועוד משתמשות בה. יש לה קהילה רחבה של מפתחים ברחבי העולם, היא מתוחזקת ובעלת דוקומנטציה נוחה.

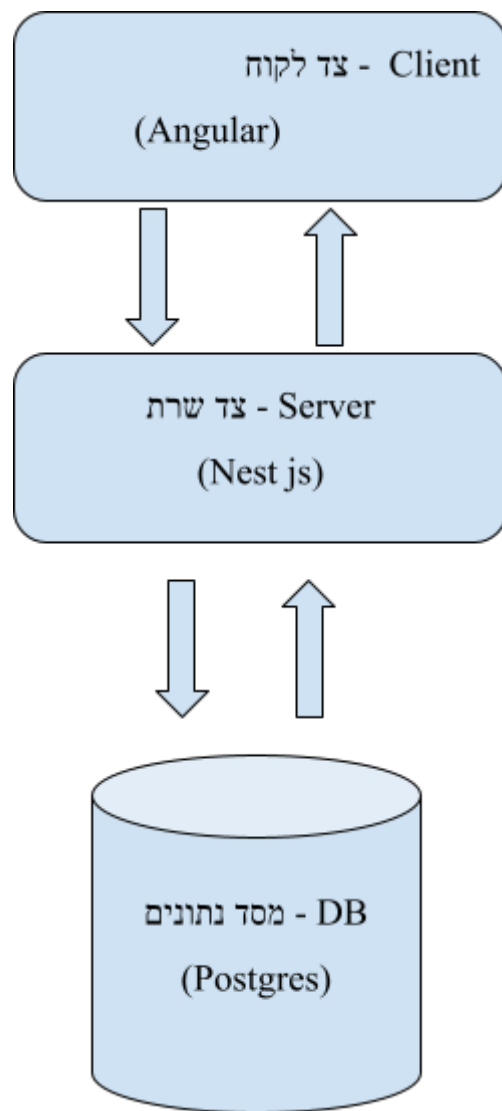
**Apollo** - ספרייה לניהול המידע בצד לקוח. שליפת המידע מהצד שרת

**Material UI** - ספריית עיצוב פופלרית, מספקת קומפוננטות מוכנות ונוחות לשימוש

**Nest.js** - ספריית פיתוח פופולרית עבור פיתוח צד שרת בשפת ג'אבה סקריפט, המספקת כלים רבים העוזרים למפתח ליצור פרויקט בקלות ובמהירות.

**GraphQL** - שפת שאילתות עבור ממשקי api, הוא מאפשר ללקוחות שפונים אליו לקחת בדיוק את חתך המידע שרוצים ולא את כל המידע וחוסך עבודה בצד השרת. במקביל מספק כלי עבודה רבי עוצמה למפתחים.

**Postgress** - פלטפורמה פופלרית למסד נתונים רציונלי שתואם את צרכי המערכת. תומכת בsql ומייחצנת פיצרים נוספים לניהול המידע.



### 5.3. שפות הפיתוח: (איזה שפות ומדוע בכמה מילים):

**Type script** - שפה מבוססת js אשר מאפשרת עבודה עם types וכך מעלה את בטיחות הקוד. Js היא אחת השפות הפופולריות לפיתוח וובי, עליה מבוססים frameworks כדוגמת angular בה אנו משתמשים.

**HTML** - שפת תכנות להצגת מידע במערכת וובית.

**SQL** - שפה למסד נתונים רציונלי. אחת השפות הפופולריות ביותר, אשר עליה מבוססות מגוון פלטפורמות כדוגמת Postgres בה אנחנו משתמשים. בנוסף, היא פשוטה וקלה להבנה ובעלת ביצועים יעילים ומהירים.

**sequelize Nest js** – נסט היא ספרייה להקמת צד שרת ts, sequelize היא ספריית שליפות שמתממשקת איתה בקלות, בה אנחנו ניעזר בכתיבה של צד השרת.

#### 5.4. תיאור הארכיטקטורה הנבחרת - הסבר בכמה מילים מדוע:

בחרנו בארכיטקטורה הנוכחית: צד לקוח, צד שרת, ומסד נתונים. צד הלקוח יהיה מה שמשתמשי המערכת רואים - מה שמאפשר למשתמשי המערכת לתקשר עם המערכת שלנו. צד השרת מנגיש לצד הלקוח את המידע ומעבד אותו. מסד הנתונים שומר לנו את כל המידע הרלוונטי.

בחרנו להשתמש בארכיטקטורה זו כדי שלא תהיה תלות בין הצדדים השונים, כל צד עומד בפני עצמו וכך במידה ונרצה לעשות refactor בצד מסויים זה לא ישפיע על הצד האחר. בנוסף ישנה חלוקה נכונה של האחריות: צד הלקוח מנגיש את המידע, והשרת מעבד אותו ומתקשר עם מסד הנתונים.

בנוסף, כיוון שכל השירותים שלנו כתובים בts החלטנו להשתמש בפלטפורמה nx.

**Nx** הינו כלי שמאפשר ניהול של כל השירותים בפרויקט אחד, mono-repo, כך שכל השירותים מאוחדים בתיקייה אחת ויש לכל הספריות אותן גרסאות. הספרייה מבצעת אופטימיזציה ומאפשרת שמירה על איכות הקוד, כמו גם תהליך ci-cd קל יותר.

#### 5.5. חלוקה לתכניות ומודולים:

##### 7.5.1 צד שרת:

**קבצי gql** - מגדירים את מבני הנתונים כפי שהם יישמרו בסכמה של השרת, כמו גם את כל הבקשות שהשרת מייחצן.

**Models** - מייצג את מבנה הטבלה כמו שמופיע במסד הנתונים

BirdInfo

BirdType

Users

Bird

Reports

**Repositories** - אחראים על ביצוע פעולות crud - create, read, update, delete

**Services** - אחראים על שמירת המידע במסד הנתונים והחזרתו במידת הצורך, מדברים עם הרפוזיטורי

**Resolvers** - מזריקים את כל הservices ובהם מוגדרות השליפות שהגדרנו בקובץ gql ואחראים לוולידציה של הרשאות

## 7.5.2 צד לקוח:

- **מסך ההתחברות:** ניתן במסך לבחור בין הרשמה להתחברות. בעת הרשמה על כל משתמש לבחור את סוג החשבון שהוא רוצה: צפר, חובב טבע, ילד. לאחר מכן יועבר למסך הראשי. במידה וכבר התחבר במכשיר בשבוע האחרון, לא יצטרך להזדהות שוב אלא יועבר ישר למסך הבית.
- **מסך הבית:** רוב המסך יהיה המפה עליה דיפולטית יוצגו כל הציפורים בישראל הנצפו על ידי המשתמשים בשבוע האחרון. במידה והמשתמש מעוניין לראות סוג ספציפי יותר, יוכל לסנן את המידע המוצג על המפה בעזרת התפריט בצד ימין. בנוסף בעת "דקירה" של נקודה על המפה ייפתח פופ-אפ למשתמש בו הוא יוכל לדווח על ציפור שראה.
- **מסך המידע:** בלחיצה על סוג ציפור כלשהו בתפריט במסך הבית, יועבר המשתמש לסוג הציפור הרלוונטי במסך המידע ושם יקבל מידע חשוב על הציפור, כמו עונות נדידה, גודל, צבע, עובדות מיוחדות ופרטים מעניינים על הציפור, כמו גם מידע שמשתמשים פרסמו על הציפור בשבוע האחרון. בנוסף במסך המידע יוכל המשתמש לצפות בכל סוגי הציפורים הקיימות באתר, ולהציע הוספה של זן חדש של ציפורים.
- **מסך הילדים:** מסך זה יופיע בתפריט רק אם המשתמש מוגדר כילד. במסך זה יהיו משחקים מותאמים לילדים בכל הגילאים – כמו טריוויה על ציפורים, משחקי מחשב שקשורים לעולם הצפרות ועוד.
- **מסך הבקשות:** מסך זה יופיע בתפריט רק אם המשתמש מוגדר כצפר. במסך זה הצפר יוכל לראות את הדיווחים של כל המשתמשים ולבחון אותם.

הצפר יהיה בעל הרשאות לאישור או סירוב הבקשות לדיווח וכך המערכת תשמור על אמינות המידע.

- 5.6. סביבת השרת (מקומי, וירטואלי, ענן, שירות אירוח): השרת ירוץ על vm באופנשיפט
- 5.7. ממשק המשתמש/לקוח - GUI: הלקוח יכנס למערכת דרך דפדפן לבחירתו – chrome, explorer וכו'
- 5.8. ממשקים למערכות אחרות\API: המערכת מתממשקת עם כלי שנקרא hasura שמתלבש עם db, מייחצן API ואחראי על ניהול ההרשאות ואבטחת המידע. hasura מקבלת מאיתנו את ההרשאות על המשתמש המתחבר ומחזירה את המידע שהוא יכול לראות לפי ההרשאות שלו.
- 5.9. שימוש בחבילות תוכנה: שימוש בחבילות תוכנה:
- 7.9.1 צד שרת: יעשה שימוש בnpm. חבילות שנשתמש בהן:
- 7.9.1.1. Nest js – ספריה להקמת שרת
- 7.9.1.2. ORM – sequeulize-ts לדיבי רלציוני
- 7.9.1.3. jwt-decode – ספריה לניהול טוקנים והרשאות
- 7.9.2 צד לקוח: יעשה שימוש בחבילות npm. העיקריות:
- 7.9.2.1. angular-material - ספריית עיצוב
- 7.9.2.2. apollo-angular - ספרייה לבקשות API
- 7.9.2.4. sweet-alert-2 - ספריה להתראות למשתמש
- 7.9.2.5. ngrx - ספריה לניהול staten בangular
- 7.9.2.6. angular-forms - ספריה לניהול נוח של שדות קלט בטפסים
- 7.9.2.7. cesium – ספריה לעבודה עם מפה

## **6. שימוש במבני נתונים**

### **6.1. נא פרט את מבני הנתונים:**

למידע הרציונלי נשתמש במסד הנתונים postgres ונשלוק אותו בצד שרת באמצעות ספריית sequelize.

6.2. נא פרט את שיטת האחסון (מאגר, קבצים ובאיזו טכנולוגיה): נשתמש בdb מסוג postgres. זהו db רלציוני שיאפשר לנו לשמור את מבני הנתונים שלנו בטבלאות וליצור קשרים לוגים ביניהם.

6.3. נא ציין מנגנוני התאוששות מנפילה/קריסה/תמיכה בטראנזקציות: אנחנו משתמשים ב kubernetes שזו פלטפורמה לניהול ופריסה של המערכת על פני קונטיינרים. בין היתר kubernetes מאפשר מגוון התאוששות אוטומטי: מאפשר ריסטארט אוטומטי במקרה של נפילה, וניטוב הבקשות לקונטיינרים הפעילים על מנת לשמור על חיות המערכת. כלומר המערכת תרוץ על מספר פודים וכך במקרה של קריסה של אחד מהם, אוטומטית המערכת תורם על הפוד הבא אחריו. וכך המשתמש לא יחוזה נפילות.

## 7. תרשימי מערכת מרכזיים

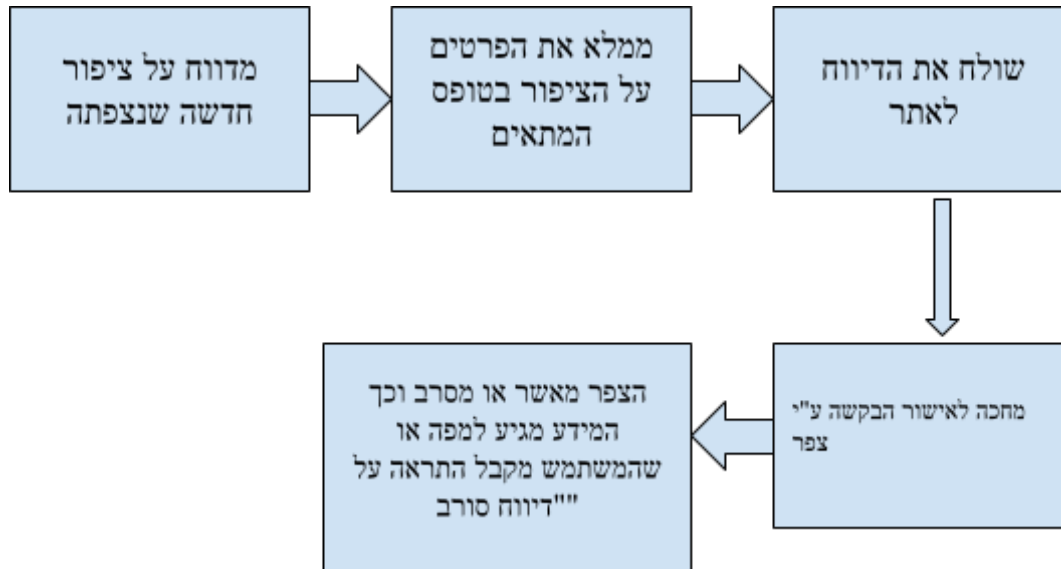
### 7.1 Use Case:

- 9.1.1: משתמש שהוא צפר יוכל לדווח על ציפור שנצפתה על ידו
- 9.1.2: משתמש שהוא צפר יוכל להוסיף סוג חדש של ציפור שלא קיים במערכת
- 9.1.3: משתמש שהוא ילד יוכל לעבור למסך הילדים ולשחק במשחקים
- 9.1.4: משתמש שהוא חובב טבע יוכל לדווח על ציפור שנצפתה על ידו, רק באישור משתמש שהוא צפר
- 9.1.5: משתמש שהוא צפר יוכל לערוך את מסך המידע על הציפורים ולהוסיף או לשנות מידע רלוונטי
- 9.1.6: חיפוש וצפייה בסוגי ציפורים
- 9.1.7: סינון סוגי הציפורים המופיעות על המפה
- 9.1.8: משתמש מכל סוג שהוא יוכל ליצור לעצמו פריסט בהתאם להעדפותיו, וכך תשופר חווית המשתמש. בעת עליית מערכת יוכל המשתמש לטעון את הפריסט, וכך מצב המפה יתעדכן בהתאם.
- 9.1.9: משתמש שהוא צפר יוכל לאשר או לסרב לדיווחים.

## 7.2 . Sequence diagram -

Client: report seeing bird, add new bird type, edit bird info, filter  
map data -> server: load preset, load map data, save bird info

## 7.3 . Data flow :



## 8. תיאור המרכיב האלגוריתמי - חישוב

המערכת תעזור לחברה להגנת הטבע לשלב את הציבור בסיוע לתחום הצפרות הישראלי על ידי שמירת מידע אמין והעברתו בין כלל המשתמשים בישראל.

המערכת תחשב לאיזה צפר מבין הצפרים במערכת הכי אידיאלי להעביר את הבקשה לדיווח, בהסתמך על תחום המחקר שלו, למשל צפר שמתעסק בנחליאלים, יגיעו אליו הבקשות הקשורות למשפחת הנחליאלים.

## 11. תיאור/התייחסות לנושאי אבטחת מידע

המערכת תנוהל ע"י הרשאות על פי סוג המשתמש ובעזרת hasura.

מידור מידע - כל צפר יבחן את הבקשות הרלוונטיות לתחומו בלבד.

משתמש שאינו צפר יצטרך לחכות לאישור הבקשה ולא יוכל לדווח סתם, אך ורק צפרים יוכלו להוסיף או לשנות מידע אודות ציפורים.

בנוסף תהיה הזדהות באמצעות jwt.

## **12. משאבים הנדרשים לפרויקט:**

### **12.1. מספר שעות המוקדש לפרויקט, חלוקת עבודה בין חברי הצוות:**

הפרויקט מתומחר בכ350 שעות עבודה.

תכנון מסד הנתונים – עדן וקטיה

מסך התחברות – קטיה

מסך המידע על הציפורים – עדן

מסך הבית – עדן וקטיה

מסך הבקשות – קטיה

מסך הילדים - עדן

עיצוב UX/UI במערכת – עדן וקטיה

בדיקות במערכת – עדן וקטיה

### **12.2. ציוד נדרש: עמדות מחשב עם windows, המחוברות לרשת צה"לית.**

### **12.3. תוכנות נדרשות:**

Webstorm :12.3.1

Git :12.3.2

Node js :12.3.3

Chrome :12.3.4

Postgress :12.3.5

Hasura :12.3.6

12.4. ידע חדש שנדרש ללמוד לצורך ביצוע הפרויקט: התממשקות עם מפה בעזרת ספריית cesium שטרם יצא לנו לעבוד איתה, בנוסף על המידע במפה להיות דינמי ולהשתנות בהתאם לזמן שחולף מזמן הדיווח.

### **12.5. ספרות ומקורות מידע:**

12.5.1. דוקומנטציה של Node

12.5.2. דוקומנטציה של Angular

12.5.3. דוקומנטציה של Angular Material

12.5.4. דוקומנטציה של Postgres

12.5.5. דוקומנטציה של Nest js

12.5.6. דוקומנטציה של Cesium

12.5.7. דוקומנטציה של Hasura

### 13. תוכנית עבודה ושליבים למימוש הפרויקט

| <u>נושא</u>                                                | <u>שעות</u> |
|------------------------------------------------------------|-------------|
| יצירת board מסודר בjira ותכנון משימות קדימה                | 5           |
| תכנון הDB, יצירת ERD                                       | 15          |
| למידת כל הספריות והטכנולוגיות הנדרשות                      | 45          |
| תכנון הארכיטקטורה וכתיבת אפיון הנדסי כולל למערכת           | 20          |
| מימוש מנגנון הרשאות בצד השרת והלקוח שינוהל ע"י hasura וjwt | 15          |

|     |                         |
|-----|-------------------------|
| 20  | מימוש צד שרת            |
| 20  | תרשים של מסכי המערכת    |
| 15  | מסך התחברות             |
| 40  | מסך הבית – מפה + תפריט  |
| 25  | מסך המידע               |
| 20  | מסך אישור/סירוב הבקשות  |
| 40  | מסך הילדים (משחקים)     |
| 10  | שליחת דיווח             |
| 10  | עריכת מידע על הציפורים  |
| 10  | אישור/סירוב דיווח       |
| 40  | העלאה לפרוד וכתובת CICD |
| 350 | סהכ                     |

#### 14. תכנון הבדיקות שיבוצעו

14.1. נא פרט בטבלה, בדיקות תהליכיות ברמת המשתמש בהן נדרשת המערכת לעמוד (full flow).

|             |       |
|-------------|-------|
| תהליך הנבדק | פירוט |
|-------------|-------|

|                     |                                                                                                                                                                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| הזדהות כצפר         | צפר הנכנס למערכת רואה את מסך הבית, ובתפריט יוצגו לו מסך המידע ומסך הבקשות – במסך המידע יוכל לערוך ולהוסיף מידע רלוונטי, ובמסך הבקשות לאשר ולשלול דיווחים הנוגעים לתחומו                                                                 |
| הזדהות כילד         | ילד הנכנס למערכת רואה את מסך הבית, בתפריט יוצג לו מסך המידע, בדומה לחובב הטבע הילד אינו בעל הרשאות לערוך את המידע אלא לצפות בלבד, בנוסף לילד יהיה גם מסך המשחקים והוא יוכל לדווח אך צריך לחכות לאישור הצפר.                             |
| הזדהות כחובב טבע    | חובב טבע הנכנס למערכת רואה את מסך הבית, בתפריט יוצג לו מסך המידע אך אינו בעל הרשאות לערוך אותו אלא לצפות בלבד. בנוסף הוא יכול לדווח אך צריך לחכות לאישור הצפר.                                                                          |
| דיווח על ציפור      | <ol style="list-style-type: none"> <li>שמירת כלל השדות שנערכו בDB</li> <li>התראה לצפר על בקשה חדשה הממתינה לאישורו</li> <li>במידה והצפר אישר – יותרע לחובב הטבע או לילד והציפור תישמר במאגר המידע. אחרת, יותרע שהבקשה נשללה.</li> </ol> |
| מנגנון סינון המפה   | סינון הציפורים על המפה לפי סוג הציפור שנבחר                                                                                                                                                                                             |
| תפריט               | בתפריט יוצגו קישורים למסכים הרלוונטיים לפי הרשאות המשתמש                                                                                                                                                                                |
| עריכת מידע על ציפור | אם המשתמש הוא צפר הוא יכול להיכנס לדף המידע ולערוך מידע על ציפורים וכך לחדש לכל המשתמשים                                                                                                                                                |

14.2. נא פרט בטבלה, מס מייצג של בדיקות יחידה למודולים המרכזיים בהן נדרשת המערכת לעמוד (unit test).

| תהליך                | פירוט                                                                                                                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| דיווח על ציפור       | לא ניתן להעביר דיווח לצפר אם לא כל שדות החובה מלאים ותקינים, תאריך: בפורמט תאריך בלבד (מכיל יום חודש ושנה, ושעות ודקות), מיקום: תתבצע בדיקה שהנ"צ אכן בישראל, סוג הציפור: בדיקה שסוג הציפור קיים במערכת |
| המתנה לאישור דיווח   | במידה וכבר דווחה הציפור על ידי משתמש כלשהו, לא יישלח דיווח חדשה על מנת למנוע כפילות מידע והעמסת יתר על הצפר. המשתמש יקבל הודעה.                                                                         |
| אישור או סירוב דיווח | אסור לאשר או לסרב דיווח יותר מפעם אחת                                                                                                                                                                   |

**15. בקרת גרסאות (version control):** שימוש בgit כדי למנוע איבוד גרסאות וכדי

להקל על עבודת הצוות ולמנוע התנגשויות בקוד.

חתימת המנחה האישי



חתימת הסטודנט



**הצהרות:**

לכבוד  
יחידת הפרויקטים  
מה"ט

**הצהרת סטודנט**

שם הסטודנט: יקר' נ. יוסף ת.ז. 326654225  
שם המכללה בה לומד הסטודנט: כסא

אני הח"מ, מצהיר בזאת כי פרויקט הנגמר וספר הפרויקט המצוייב נעשו על ידי בלבד.  
פרויקט הנגמר נעשה על סמך הנשאים שלמדתי במכללה ובאופן עצמאי.  
פרויקט הנגמר וספר הפרויקט נעשו על בסיס הנחייתו של המנחה האישי.  
מקורות המידע בהם השתמשתי לביצוע פרויקט הנגמר מצוינים ברשימת המקורות המצוינים  
בספר הפרויקט.  
אני מודע לאחריות שהני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנאמר בה אמת ורק  
אמת.

תאריך: 03/03/2025

חתימת הסטודנט: 

**אישור המנחה האישי**

הריני מאשר שהפרויקט בוצע בהנחייתי, קראתי את ספר הפרויקט ומצאתי כי הוא מוכן לצורך  
הגשת הסטודנט להגנה על פרויקט נגמר.

שם המנחה: ד"ר ע' ע' ע' חתימה:  תאריך: 03/03/2025

**אישור ראש המגמה**

הריני מאשר שספר הפרויקט מוכן לצורך הגשת הסטודנט להגנה על פרויקט הנגמר.

שם ראש המגמה: \_\_\_\_\_ חתימה: \_\_\_\_\_ תאריך: \_\_\_\_\_

עודכן (10/17)

דרך מנחם בגין 86 תל אביב ת.ד. 36049 מיקוד 67138  
טלפון: 03-7347521 פקס: 03-7347644

לכבוד  
יחידת הפרויקטים  
מחייט

**הצהרת סטודנט**

שם הסטודנט: 120 136  
שם המכללה בה לומד הסטודנט: ת.י. 325879666  
נסר

אני החיימ, מצהיר בזאת כי פרויקט הגמר וספר הפרויקט המצייב נעשו על ידי בלבד.  
פרויקט הגמר נעשה על סמך הנשאים שלמדתי במכללה ובאופן עצמאי.  
פרויקט הגמר וספר הפרויקט נעשו על בסיס הנחייתו של המנחה האישי.  
מקורות המידע בהם השתמשתי לביצוע פרויקט הגמר מצוינים ברשימת המקורות המצוינים בספר הפרויקט.  
אני מודע לאחריות שהני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנאמר בה אמת ורק אמת.

חתימת הסטודנט:   
תאריך: 03/03/2025

**אישור המנחה האישי**

הריני מאשר שהפרויקט בוצע בהנחייתי, קראתי את ספר הפרויקט ומצאתי כי הוא מוכן לצורך הגשת הסטודנט להגנה על פרויקט גמר.

שם המנחה: ד"ר ד' ע' 18 חתימה:  תאריך: 03/03/2025

**אישור ראש המגמה**

הריני מאשר שספר הפרויקט מוכן לצורך הגשת הסטודנט להגנה על פרויקט הגמר.

שם ראש המגמה: \_\_\_\_\_ חתימה: \_\_\_\_\_ תאריך: \_\_\_\_\_



### **הוקרת תודה:**

אנו רוצים להוקיר תודה למנחה המקצועי שלנו, עילי, שליווה אותנו לאורך כל הדרך ועזר לנו עם כל שאלה מקצועית, ארכיטקטונית ואלגוריתמית ונענה לכל בקשה שהייתה לנו באדיבות ומסירות.

כמו כן, נרצה להודות לבסמ"ח שסיפקו לנו חומרים והדרכה לפרויקט וכן סיפקו משוב להצעת הפרויקט ועזרו לנו להגיע לתוצר הנ"ל.

## **תקציר העבודה**

מרכז הצפרות הישראלי פועל למען שמירה על מגוון מיני העופות בישראל, שימור ושיקום בתי הגידול החשובים לקיומם. אנו שואפים לחזק את התמיכה הציבורית בשמירת הטבע בישראל על ידי שיתוף פעולה של הציבור ביחד עם המדינה על מנת לשמור על כל מיני הציפורים בישראל. מדי שנה, אלפי זנים של ציפורים מבקרים את ארצנו היפה, ועדיין אין דרך לדווח היכן הן נמצאות, לאן פניהן מועדות ולגלות היכן נמצאים הזנים הנדירים.

המערכת תנגיש פלטפורמה לניהול כל זני הציפורים בישראל, בה אפשר לדווח על נוכחות ציפורים במקום מסוים, כמו גם להוסיף פירוט במידה ומדובר בזן נדיר, וכך הציבור יגלה יותר ערנות כלפי הטבע בישראל ויתחבר יותר לעולם הצפרות שהולך ודועך עם השנים.

במערכת מספר מסכים – מסך התחברות, הכולל הרשמה ובחירת סוג משתמש, מסך הבית של המערכת שיכיל מפה ועליה הציפורים שדווחו לאחרונה, מסך המידע – יכיל מידע על זנים נדירים. ומסך הילדים – ינגיש משחקי למידה לילדים (יופיע רק אם המשתמש הוא ילד). מסך הבקשות – בו ניתן לצפות בבקשות שדיווחתי על ציפורים.

## עקרונות בסיסיים וכללים מנחים בפרויקט

בכתיבה ובמימוש של הפרויקט התבססנו על כמה עקרונות חשובים.

### (1) קריאות

הקפדנו על כתיבת קוד מסודר, נקי וקריא. בכל פעם שאחד מאיתנו סיים משימה, השני עבר לו על הקוד ובחן אותו על מנת שיהיה הכי קריא שאפשר. כך, ככל שהתקדמנו בפרויקט היה קל להבין מה עשינו בשלבים המוקדמים יותר - ולפתח בקלות.

### (2) גמישות

כתבנו את הקוד בצורה גמישה, כזו שמאפשרת הסתגלות לשינויים במידה ויש צורך. לא השתמשנו בדברים שדורשים תלות מהותית אחד בשני, וכך קיבלנו קוד גמיש שאפשר לנו לשדרג ולהרחיב את המערכת בצורה קלה

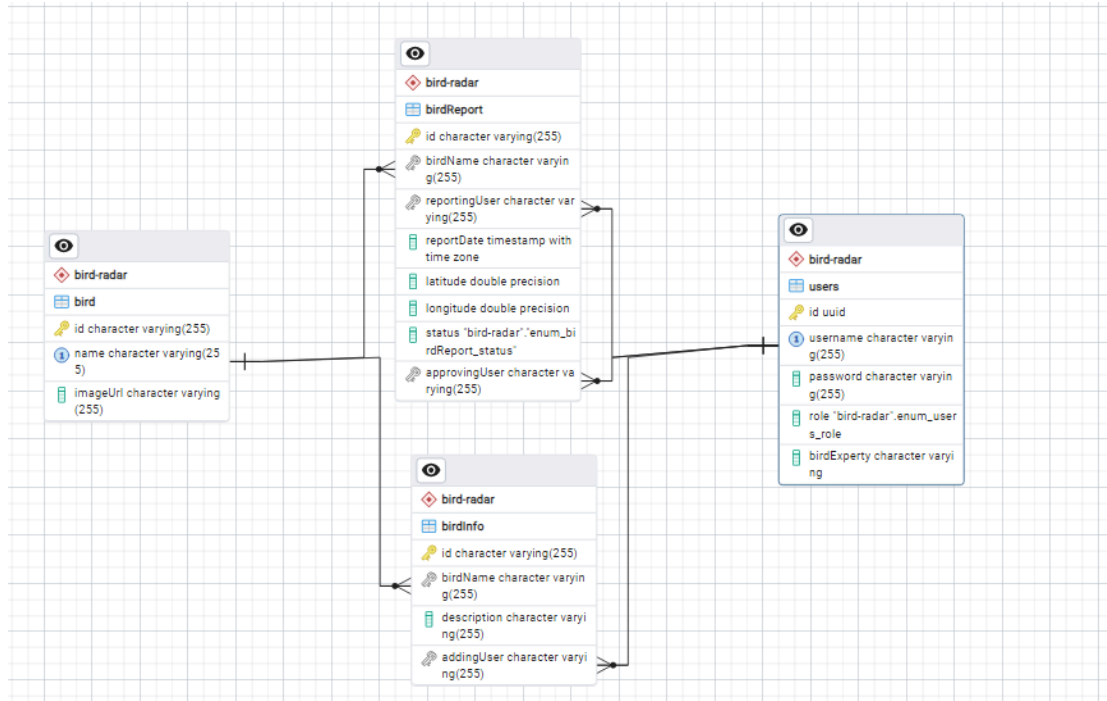
### (3) הפרדה ל-micro-services

אמנם השתמשנו במונו-רפו NX אבל הפרדנו את השירותים שלנו לפי הלוגיקה ולפי הצורך. הפרדנו לצד לקוח שנכתב באנגולר, ולצד שרת שנכתב ב-nest. עם זאת, השתמשנו בספרייה משותפת לכל השירותים שתכיל דברים שכולם צריכים models - כך types במערכת יהיו תואמים בכל micro service. חשוב מאוד להפריד כדי לא להכביד ולא ליצור תלויות אחד בשני.

### (4) גנריות

מנענו חזרה על קוד ושמרנו על עיקרון הגמישות בכך שכתבנו קומפוננטות בצורה גנרית. כתבנו את הקומפוננטות בכלי שנקרא storybook ומאפשר לנו לכתוב ui בצורה אחידה, ניתן להריץ אותו בלי תלות בclient ולראות את הקומפוננטות השונות.

## תרשים ERD



## צד שרת

את צד השרת כתבנו ב-nestjs, והשתמשנו בgraphql על מנת להזרים את המידע.

צד השרת שלנו מחולק לשכבות, השכבות קיימות לכל ישות רלוונטית במערכת:

Models + Gql

Services

Resolvers

### שכבת Resolvers

שכבת ה-Resolver אחראית על עיבוד ושירות בקשות graphql. היא מטפלת בבקשות query, mutation, ו-subscription. בקשה שמגיעה מהלקוח מגיעה קודם כל אליה, והיא מעבירה את הבקשה לservice. לאחר מכן המידע מוחזר ללקוח. בgraphql הבקשה תמיד מופנית לאותה כתובת, הכתובת של השרת, והיא תמיד בקשת http מסוג POST, וגוף הבקשה מכיל בעצם את התוכן שלה (יכול להיות query - בקשת מידע, או mutation - שינוי מידע). בנוסף, במידה ויש לזרוק שגיאה השגיאה תיזרק בשכבה זו ותוחזר ללקוח.

### שכבת Services

שכבת ה-Services, נקראת לפעמים גם Buisness layer, מכילה את הלוגיקה העסקית של האפליקציה. כלומר, כל חישוב או פעולה לוגית שצריכה לקרות בשרת תקרה בשכבה זו. בשכבה זו נזריק גם Repository - שהוא אחראי על תקשורת עם מסד הנתונים ומטפל בכל פעולות ה-CRUD (יצירה, קריאה, עדכון ומחיקה) בבסיס הנתונים בצורה יעילה ואמינה.

### שכבת Models + gql

שכבת המודלים, אחראית על מידול המידע כפי שיישמר בDatabases, מבחינת עמודות. כאן נגדיר איך כל ישות שלנו קיימת אל מול הטבלה שלה בדיבי. אחרי ההגדרה נוכל לכתוב גם את הסכמה והטבלה המתאימה בDB, ולקבל את repository שאותו נוכל להזריק בשכבת service.

gql:

שניתן לעשות על אותה ישות, כמו query וmutation קובץ בו מוגדרות כל הפעולות גם כל הטיפים שלה. מכאן בעצם השרת שלנו מקבל את כל הפעולות שהוא יכול לייחצן, והמימוש שלהן בפועל מתבצע בשכבות השונות

### יתרונות החלוקה לשכבות:

## היתרונות של ארכיטקטורת שכבות הם:

1. כל שכבה אחראית על תחום אחר. גישה זו מפחיתה תלות בין החלקים השונים של המערכת ומאפשרת לכל שכבה להתמקד בתפקידה הייחודי.  
החלוקה הזו מפשטת את הפיתוח ומאפשרת הבנה קלה יותר של כל חלק במערכת.
2. תחזוקה קלה יותר: קל לתקן באגים כי כל שכבה מטפלת בנושא אחר. לכן קל לבודד את מקור הבאג.
3. המערכת גמישה יותר לשינויים ולהוספת פיצ'רים חדשים מבלי לגרום לשינויים בשכבות אחרות. כל ישות חדשה תקבל שכבות משלה ולא "תיגע" בשכבות אחרות.  
אין ערבוב בין דברים לא קשורים.

## מרכזים במערכת API

### UserResolver

מנגיש את הפעולות שניתן לבצע על משתמש במערכת - התחברות, הרשמה, קבלת משתמש לפי id (לחיבור חוזר בעזרת token)

```
@Resolver() Show usages 1 edens *
export class UserResolver {
  constructor( no usages 1 edens
    private userService: UserService,
    private authService: AuthService
  ) {}

  @Query() no usages 1 edens
  async login(
    @Args('username') username: string,
    @Args('password') password: string
  ): Promise<string> {
    const user: UserModel = await this.userService.findOneByName(username);

    if (user?.password !== password) {
      throw new UnauthorizedException('שם משתמש או סיסמה שגויים');
    }

    return this.authService.generateToken(user);
  }
}
```

```
@Mutation('signup') no usages 1 edens
async signUp(@Args('user') user: UserModel): Promise<string> {
  const newUser: UserModel = await this.userService.create(
    user.username,
    user.password,
    user.role
  );

  if (!newUser) {
    throw new UnauthorizedException(
      `אנא בחר שם אחר ${user.username} קיים כבר משתמש עם השם`
    );
  }

  return this.authService.generateToken(newUser);
}

@Query() no usages 1 edens
async getUserById(@Args('userId') id: string): Promise<UserModel> {
  const user: UserModel = await this.userService.findOneById(id);

  if (!user) {
    throw new UnauthorizedException(
      `username with id ${id} does not exist, please try again`
    );
  }
}
```

## BirdResolver

מנגיש הוספת סוג ציפור (רק למשתמש בעל הרשאות מתאימות), וקבלת כל הציפורים

```
@Resolver() Show usages  ⚙️ edens
export class BirdResolver {
  constructor( no usages  ⚙️ edens
    private birdService: BirdService,
    private userService: UserService,
  ) {}

  @UseGuards(AuthGuard) no usages  ⚙️ edens
  @Mutation()
  async addBird(@Args('bird') bird: BirdInput, @Context('user') userData: any): Promise<BirdModel> {
    const user : UserModel = await this.userService.findOneByName(userData.username);

    if (!user) {
      throw new UnauthorizedException(
        'no user, cannot add a new bird'
      );
    }

    if (
      !(user.role === UserRoleEnum.ADMIN)
    ) {
      throw new UnauthorizedException(
        'you don"t have permissions to add a new bird'
      );
    }

    return await this.birdService.createBird(bird);
  }
}
```

```
@UseGuards(AuthGuard) no usages  ⚙️ edens
@Query()
async getAllBirds(): Promise<BirdModel[]> {
  return await this.birdService.findAllBirds();
}

@UseGuards(AuthGuard) no usages  ⚙️ edens
@Query()
async getBirdById(@Args('id') id: string): Promise<BirdModel> {
  return await this.birdService.findById(id);
}
```

## BirdReportResolver

מנגיש פעולות שקשורות לדיווח על ציפור - הוספת דיווח, אישור דיווח, דחיית דיווח, קביעת משתמש מאשר, קבלת כל הדיווחים הרלוונטיים למפה (24 שעות אחרונות)

```
export class BirdReportResolver {  
  }  
  
  @UseGuards(AuthGuard)  no usages  1 edens  
  @Mutation()  
  async addReport(  
    @Args('report') report: BirdReportInput,  
    @Context('user') userData: any  
  ): Promise<string> {  
    const user: UserModel = await this.userService.findOneByName(userData.username);  
  
    if (!user) {  
      throw new UnauthorizedException(  
        `no such user: ${userData.username}, cannot report the bird`  
      );  
    }  
  
    const bird: BirdModel = await this.birdService.findByName(report.birdName);  
  
    if (!bird) {  
      throw new UnauthorizedException(  
        `no such bird exists: ${report.birdName}`  
      );  
    }  
  
    const isAuthorizedToApprove: boolean =  
      user.role === UserRoleEnum.ADMIN ||  
      user.role === UserRoleEnum.BIRD_WATCHER;  
  
    const newReport: BirdReportModel = isAuthorizedToApprove  
      ? await this.birdReportService.createApprovedReport(report, user.username)  
      : await this.birdReportService.createPendingReport(report, user.username);  
  
    return newReport.id;  
  }  
}
```

## BirdInfoResolver

מנגיש דברים שקשורים למידע על הציפורים - מידע שמשתמשים יכולים להוסיף

```
export class BirdInfoResolver {
  @UseGuards(AuthGuard) no usages  ⚡ edens
  @Mutation()
  async addBirdInfo(@Args('birdInfo') birdInfo: BirdInfoInput, @Context('user') userData: any): Promise<BirdInfoModel> {
    const user: UserModel = await this.userService.findOneByName(userData.username);

    if (!user) {
      throw new UnauthorizedException(
        'no user, cannot add a new bird info'
      );
    }

    if (
      !(user.role === UserRoleEnum.ADMIN || user.role === UserRoleEnum.BIRD_WATCHER)
    ) {
      throw new UnauthorizedException(
        'you don't have permissions to add an info to a bird'
      );
    }

    return await this.birdInfoService.createBirdInfo(birdInfo, user.username);
  }
}
```

```
@UseGuards(AuthGuard) no usages  ⚡ edens
@Query()
async getAllBirdInfosGroupedByBirds(): Promise<BirdInfoGroup[]> {
  const birdInfos: BirdInfoModel[] = await this.birdInfoService.findAllBirdInfos();
  const grouped: any = groupBy(birdInfos, 'birdName');
  // @ts-ignore
  return Object.entries(grouped).map(([key, items]: [string, unknown]) => ({ birdName: key, infos: items }));
}

@UseGuards(AuthGuard) no usages  ⚡ edens
@Query()
async getBirdInfoById(@Args('id') id: string): Promise<BirdInfoModel> {
  return await this.birdInfoService.findById(id);
}
```

## חלק אלגוריתמי חישובי בצד שרת

במערכת, אנחנו רוצים לנהל את העומסים על הצפרים בצורה יעילה. משמע, כשנוצרת בקשה חדשה לדיווח עלינו למצוא את הצפר הכי מתאים לניהול הבקשה.

בטבלת המשתמשים יש לכל משתמש מסוג צפר שדה `birdExpert` (לשאר המשתמשים השדה `null`). ראשית מוצאים את הבקשה הרלוונטית ומחלצים ממנה את הציפור שהיא עליה.

לפי הציפור מחלצים את כל הצפרים שמתמחים בציפור הזו. עוברים צפר צפר ולכל צפר מוצאים את מספר הבקשות שיש לו לאשר. המטרה היא למצוא את הצפר המתאים שיש לו את הכמות הקטנה ביותר של הבקשות - על מנת לנהל עומסים בצורה יעילה ולא להכביד על אף צפר.

```
@UseGuards(AuthGuard) no usages new *
@Query()
async matchBirdReportToBirdObserver(id: string) : Promise<{ expert: UserModel; reportsAmount: number }> {
  const report : BirdReportModel = await this.birdReportService.findById(id);
  const bird : string = report.birdName;
  const experts : UserModel[] = await this.userService.findAllExpertsByBirdName(bird);
  let minReports : { expert: UserModel; reportsAmount: number } = {
    expert: experts[0],
    reportsAmount: (await this.birdReportService.findAllReportsToApprove(experts[0])).length
  }
  for (const expert of experts) {
    const report = {
      expert,
      reportsAmount: (await this.birdReportService.findAllReportsToApprove(expert)).length
    };
    if (report.reportsAmount < minReports.reportsAmount) {
      minReports = report;
    }
  }
  return minReports;
}
```

## הזדהות והרשאות

על כל פונקצייה בresolver וכן על כל פעולה בצד הלקוח מוגדרת הרשאה הרלוונטית לה, כך ניתן לשלוט על הפעולות המתבצעות במערכת.

בצד השרת אנו מנהלים את ההרשאות ע"י JWT ושמירת סוג המשתמש בDB.

כאשר משתמש נכנס למערכת בודקים אם יש לו token, אם אין לו token בתוקף הוא לא יוכל להיכנס וינותב חזרה לעמוד ההתחברות. המערכת מותאמת לכל סוג משתמש - כל אחד יראה את הדברים שרלוונטים לו בלבד.

הזדהות: את ההזדהות על כל בקשה בשרת נבצע בעזרת AuthGuard - ניתן לראות בבקשות AuthGuard - UseGuards:

```
@Injectable() Show usages 1 edens
export class AuthGuard implements CanActivate {
  constructor(private jwtService: JwtService) {} no usages 1 edens

  async canActivate(context: ExecutionContext): Promise<boolean> { no usages 1 edens
    const ctx: GqlExecutionContext = GqlExecutionContext.create(context);
    const request: any = ctx.getContext().req;
    const token: string = this.extractTokenFromHeader(request);
    if (!token) {
      throw new UnauthorizedException();
    }
    try {
      ctx.getContext().user = await this.jwtService.verifyAsync(
        token,
        {
          secret: process.env.JWT_KEY
        }
      );
    } catch {
      throw new UnauthorizedException();
    }
    return true;
  }

  private extractTokenFromHeader(request: Request): string | undefined { Show usages 1 edens
    const [type, token] = request.headers['authorization']?.split(' ') ?? [];
    return type === 'Bearer' ? token : undefined;
  }
}
```

מחלצים מהבקשה את Headers שלה ובודקים אם יש token - אם הוא קיים מוודאים שהוא מאומת בעזרת הsecret שהגדרנו. כך נדע אם הtoken נוצר אצלנו או במקום אחר. רק אם הפונקציה canActivate תעבור ותחזיר אמת, המשתמש יוכל להמשיך בבקשה.

בכל בקשה אחרי שמקבלים את הtoken - ניתן לשלוף את המשתמש לפי UserId ולקבל את ההרשאות שלו לפי התפקיד שלו.

יש ארבעה תפקידים עיקריים:

משתמש מסוג זה בעל הרשאות דיווח אך לא יכול לאשר את - CHILD  
הבקשות, כמו כן יש לו מסך נפרד של משחקים חינוכיים לילדים

חובב טבע, משתמש מסוג זה בעל הרשאות לדיווח אך - NATURE\_LOVER  
צריך שיאשרו לו את הבקשות, הוא יכול לצפות בכל המידע אודות הציפורים  
אך אינו בעל היכולת לערוך אותו

צפר, משתמש מסוג זה בעל הרשאות לאשר דיווחים - BIRD\_WATCHER  
רק כאלה הנוגעים לתחום המומחיות שלו. הוא יכול לדווח בלי לחכות לאישור,  
דיווח שלו יופיע מיד על המפה. כמו כן, הוא יכול לערוך מידע אודות הציפורים  
ולהוסיף מידע

מנהל, משתמש על במערכת. יכול לבצע את כל הפעולות - הוספת - ADMIN  
דיווח, הוספת ציפור, הוספת מידע ועריכתו. אף מסך המשחקים גלוי בפניו

## התממשקות מול הDB

התממשקנו מול הדיבי בעזרת ספריית sequelize, ספריית ORM כלומר דרך לשמירת מודלים של מידע בצד השרת כפי שהם נמצאים במסד הנתונים. בונים לכל טבלה מודל בצד השרת ואז מקבלים repository שמייחצן פעולות - CRUD create, read, update, delete

ישויות:

User

```
@Table({ tableName: 'users', schema: 'bird-radar' }) Show usages ⚙️ edens *
export class UserModel extends Model {
  @Column({
    type: DataType.UUID,
    allowNull: false,
    unique: true,
    primaryKey: true,
    defaultValue: UUIDV1(),
  })
  id: string;

  @Column({
    type: DataType.STRING,
    unique: true,
    allowNull: false,
  })
  username: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
  })
  password: string;

  @Column({
    type: DataType.ENUM('NATURE_LOVER', 'ADMIN', 'CHILD', 'BIRD_WATCHER'),
    allowNull: false,
    defaultValue: UserRoleEnum.NATURE_LOVER,
  })
  role: UserRoleEnum;

  @Column({
    type: DataType.STRING,
    allowNull: true,
    defaultValue: null,
  })
  birdExpert: string;
}
```

ניתן לראות שיש קשר בין הטבלאות - BirdInfo

מקושר לטבלת הציפורים בקשר של One to One, ומקושר לטבלת users בקשר של One to One

```
@Table({ tableName: 'birdInfo', schema: 'bird-radar' }) Show usages
export class BirdInfoModel extends Model {
  @Column({
    type: DataType.STRING,
    allowNull: false,
    defaultValue: UUIDV1(),
    primaryKey: true,
  })
  id: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
    references: {
      model: BirdModel,
      key: 'name'
    },
    onDelete: 'CASCADE'
  })
  birdName: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
  })
  description: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
    references: {
      model: UserModel,
      key: 'username'
    },
    onDelete: 'CASCADE'
  })
  addingUser: string;
}
```

## BirdReport

מקושר לטבלת הציפורים בקשר  
one to one ולטבלת המשתמשים  
one to one של בקשר

```
@Table({ tableName: 'birdReport', schema: 'bird-radar' }) Show usages
export class BirdReportModel extends Model {
  @Column({
    type: DataType.STRING,
    allowNull: false,
    defaultValue: UUIDV1(),
    primaryKey: true,
  })
  id: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
    references: {
      model: BirdModel,
      key: 'name'
    },
    onDelete: 'CASCADE'
  })
  birdName: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
    references: {
      model: UserModel,
      key: 'username'
    },
    onDelete: 'CASCADE'
  })
  reportingUser: string;

  @Column({
    type: DataType.DATE,
    allowNull: false,
    defaultValue: new Date(),
  })
  reportDate: Date;
}
```

```
@Column({
  type: DataType.DOUBLE,
  allowNull: false,
})
latitude: number;

@Column({
  type: DataType.DOUBLE,
  allowNull: false,
})
longitude: number;

@Column({
  type: DataType.ENUM('PENDING', 'APPROVED', 'REJECTED'),
  allowNull: false,
  defaultValue: ReportStatusEnum.PENDING,
})
status: ReportStatusEnum;

@Column({
  type: DataType.STRING,
  allowNull: true,
  references: {
    model: UserModel,
    key: 'username'
  },
})
reporterUsername: string;
```

## Bird

```
@Table({ tableName: 'bird', schema: 'bird-radar' }) Show us
export class BirdModel extends Model {
  @Column({
    type: DataType.STRING,
    allowNull: false,
    defaultValue: UUIDV1(),
    primaryKey: true,
  })
  id: string;

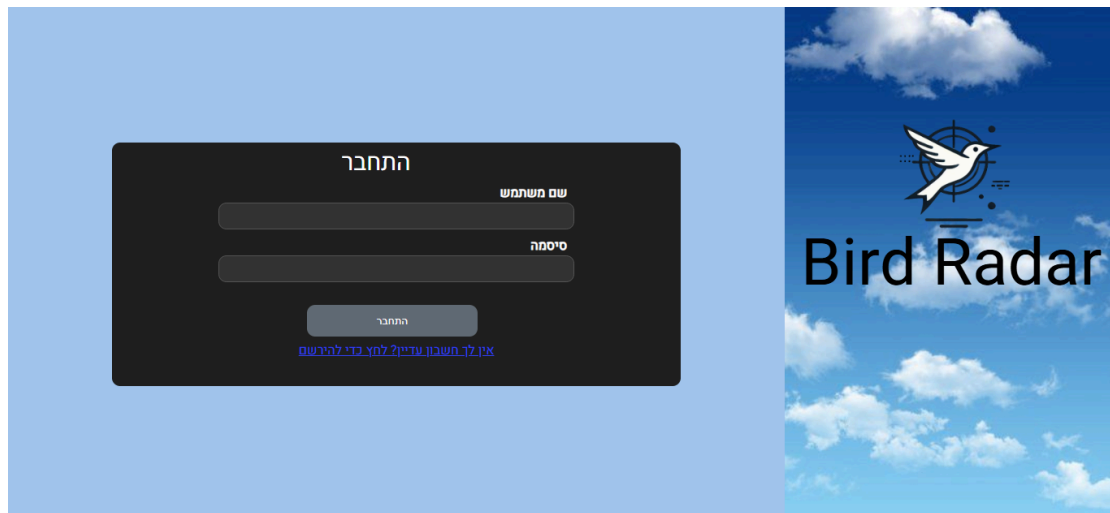
  @Column({
    type: DataType.STRING,
    unique: true,
    allowNull: false,
  })
  name: string;

  @Column({
    type: DataType.STRING,
    allowNull: true,
  })
  imageUrl?: string;
}
```

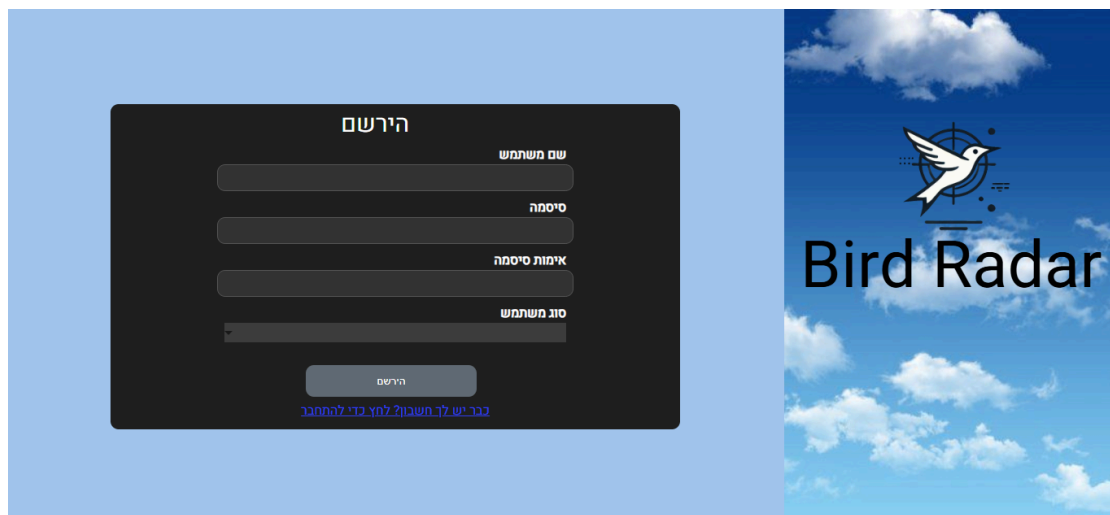
לכל מודל שהגדרנו ניתן להזריק Repository שאנו משתמשים בו ב Services על מנת לקבל את הפעולות הבסיסיות שצריך לבצע על כל ישות.

## צד לקוח

מסכים מרכזיים במערכת - מסך התחברות (במידה והמשתמש קיים):



בעת כניסה זה המסך הנראה למשתמש - במידה ואין לך משתמש תוכל ללחוץ על הקישור הכחול ותגיע למסך ההרשמה:



אם קיים משתמש, תוכל להתחבר ותגיע למסך הבית. במידה ואין, עליך להירשם ולבחור את סוג המשתמש שלך. במידה ותבחר צפר - עליך לספק דיפלומה מתאימה שתוכיח כי אתה צפר. רק כאשר מנהל יאשר את הדיפלומה תקבל את הרשאת הצפר, עד אז תחובר למערכת כחובב טבע.

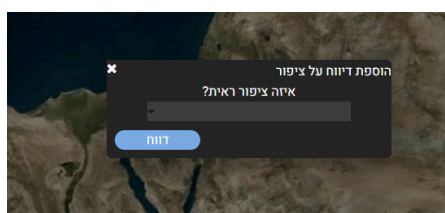
## מסך הבית

בהתחברות תגיע למסך הבית:

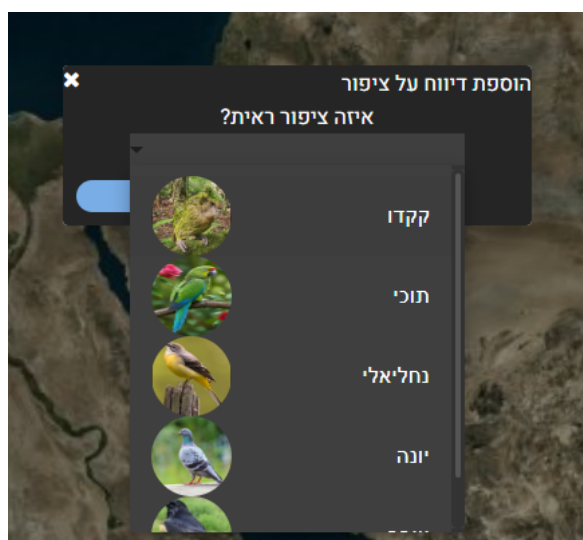


כרגע, המפה ריקה כי אין דיווחים על ציפורים ביממה האחרונה. בצד ימין יש תפריט עם פעולות שניתן לבצע - דיווחים, מעבר לעמוד המידע, מעבר לעמוד הבקשות, והתנתקות (תחזיר למסך ההתחברות).

פעולה מרכזית במערכת - הוספת דיווח על ציפור - בעת לחיצה על הכפתור ואז דקירת נ"צ במפה יפתח הטופס הבא:



ועל המשתמש לבחור מהציפורים הקיימות במערכת:

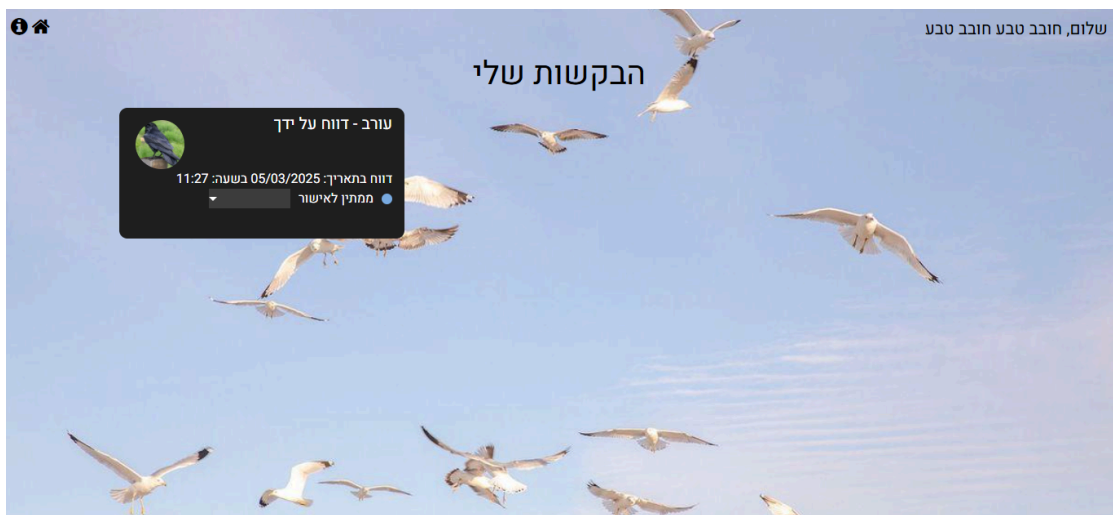


לאחר בחירת ציפור ולחיצה על דיווח - במידה והמשתמש צפר או מנהל, הציפור תתווסף אוטומטית למפה והבקשה תאושר - לאחר הוספת הציפור המפה תיראה באופן הבא:



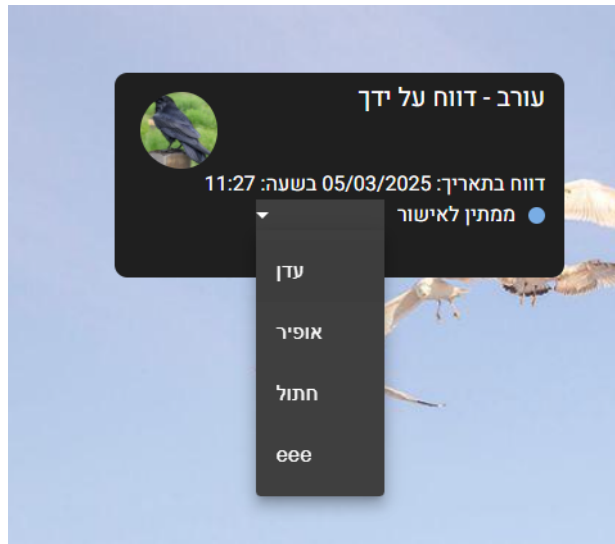
הציפור התווספה למפה וניתן לראות מי המשתמש שדיווח עליה (המפה תתעדכן באותו רגע לכל המשתמשים), בנוסף מכיוון שהמשתמש המחובר הוא המדווח הציפור תופיע בתפריט.

במידה והמשתמש אינו צפר או מנהל, כלומר ילד או חובב טבע, הציפור לא תתווסף אוטומטית אלא תיפתח בקשה ממתינה לאישור הדיווח. המשתמש המדווח יקבל הודעה שהבקשה ממתינה ועליו לבחור צפר או מנהל מאשר בעמוד הבקשות, זה ייראה כך:

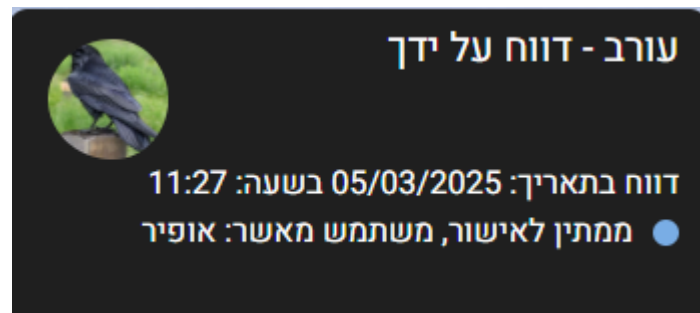


ניתן לראות את הבקשה שדווחה על ידך, ועליך לבחור משתמש שיאשר את הבקשה. המשתמש הינו מסוג צפר שמומחה בתחום הציפור עליה דווח או מנהל, ויוצגו לך רק 5 הצפרים עם הכמות בקשות הכי קטנות (שוב על מנת לנהל עומסים).

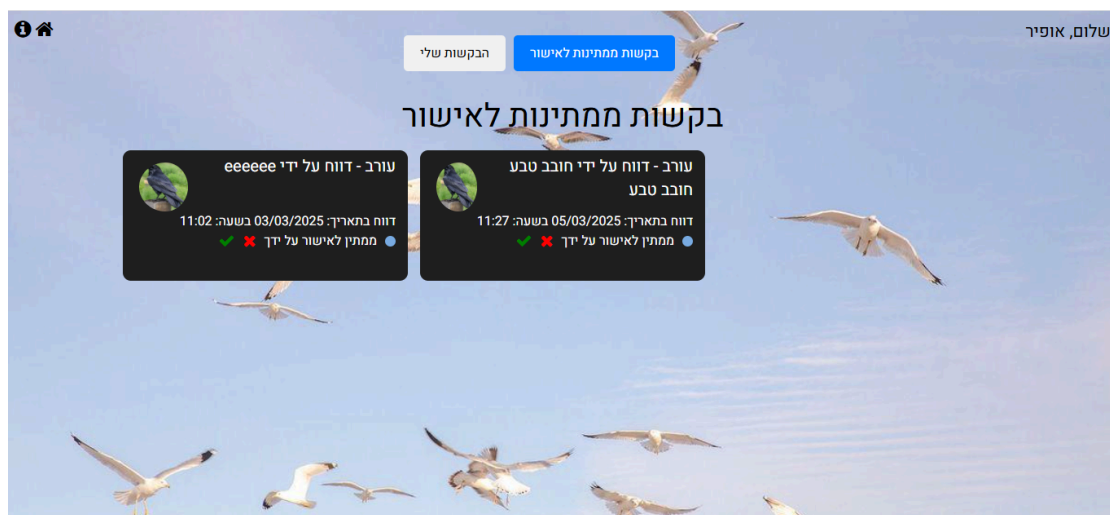
עליך לבחור משתמש מאשר:



לאחר הבחירה זה ייראה כך:



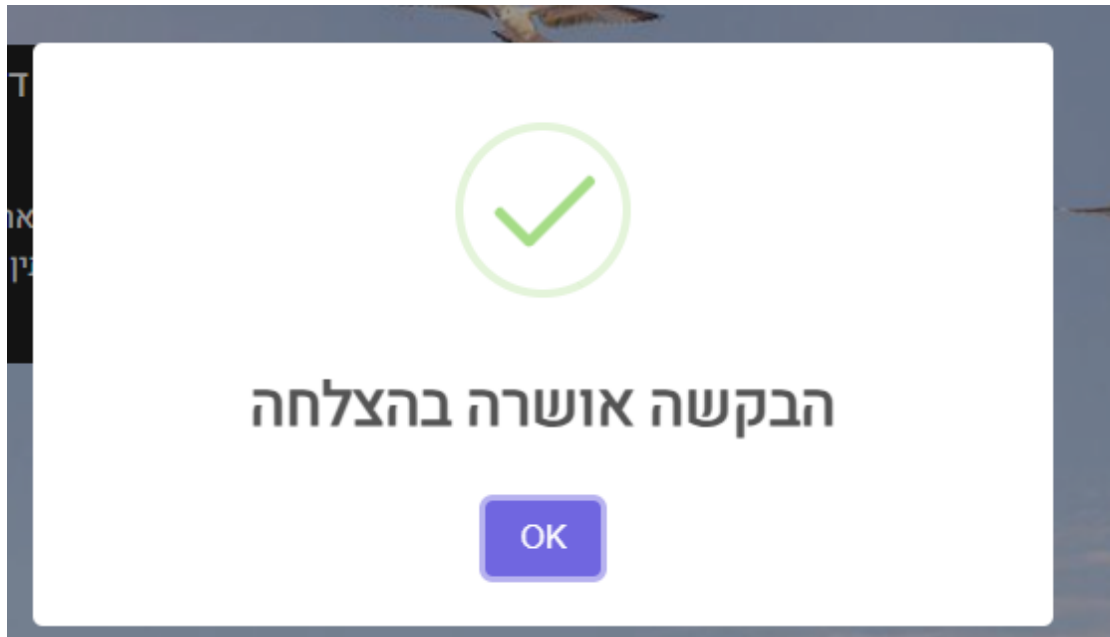
המשתמש המאשר "אופיר" יקבל הודעה שעליו לאשר את הבקשה. במסך הבקשות של אופיר, זה ייראה כך:



ניתן לראות שלאופיר, מכיוון שהיא צפרית, יש שני עמודים במסך הבקשות -  
הבקשות שלה (אלו מאושרות אוטומטית), ובקשות ממתינות לאישור - בקשות  
שמשתמשים אחרים ביקשו שאופיר תאשר.

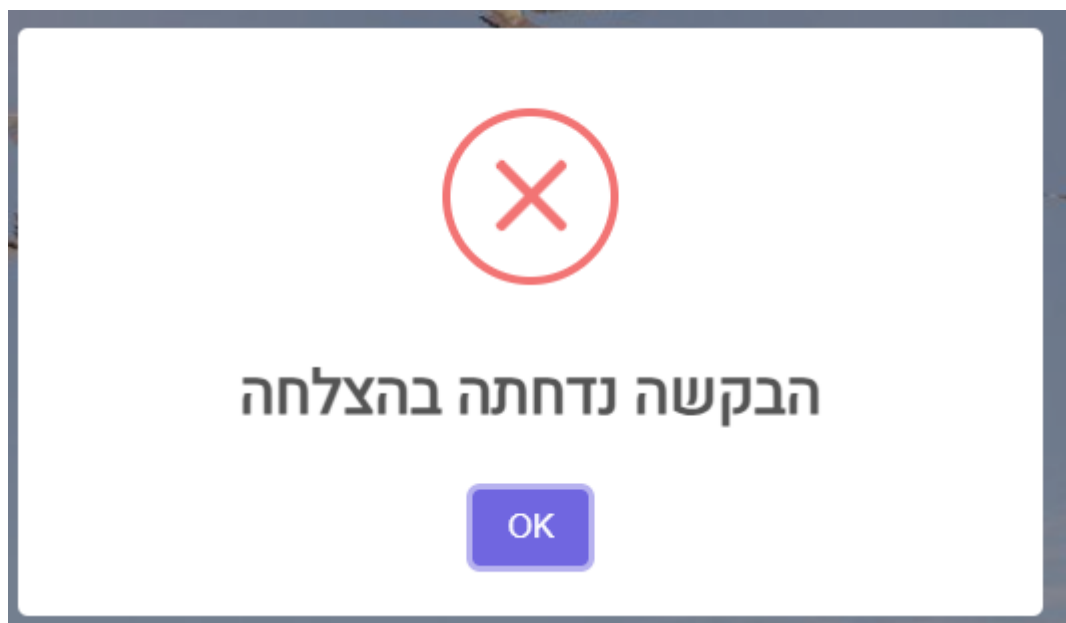
אופיר, בתור צפרית מומחית של משפחת העורבים, יכולה לבחון את הבקשות ולקבל  
החלטה האם לאשר או לדחות את הבקשה (לפי שיקולי מיקום, תקופת נדידה וכו').

לאחר לחיצה על אישור, יתקבל:



והמשתמש שבקשתו אושרה יקבל הודעה.

לעומת זאת, בעת דחייה תתקבל ההודעה:

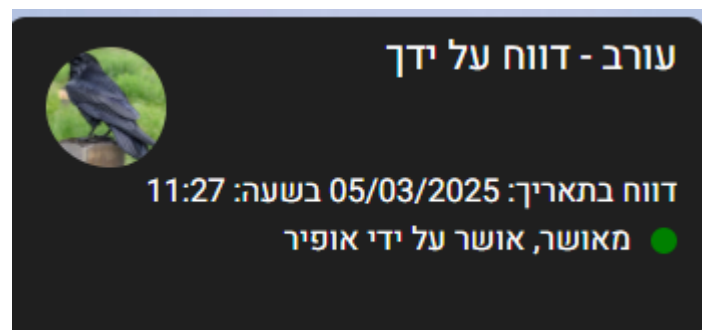


והמשתמש יקבל הודעה שבקשתו נדחתה.

לאחר אישור הבקשה על ידי אופיר, חובב טבע קיבל הודעה שדיווחו אושר והמפה התעדכנה, כמו גם הדיווחים שלו:

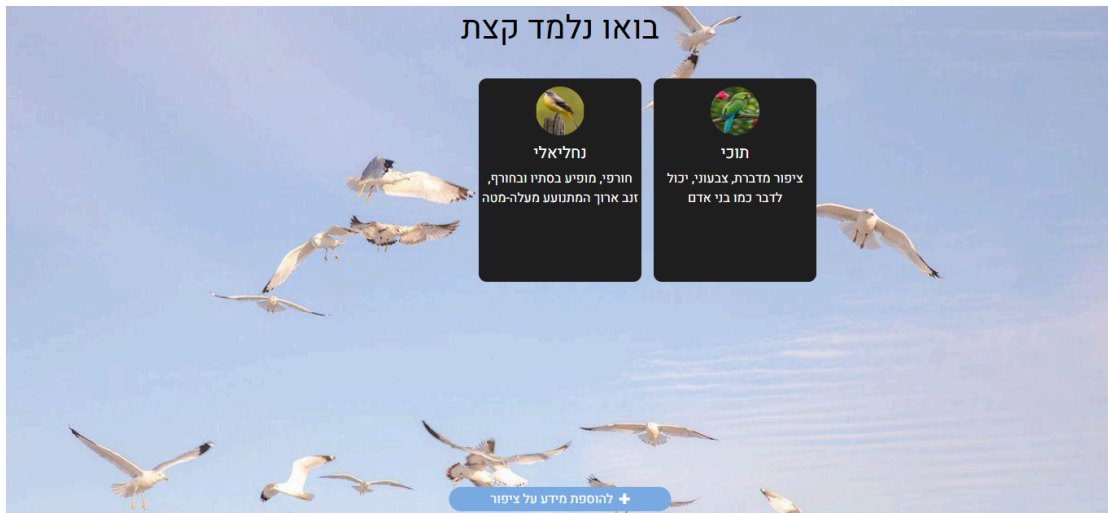


בנוסף הוא יכול לראות במסך הבקשות שהבקשה אושרה על ידי אופיר:



## מסך המידע

במסך זה ניתן לראות מידע אודות סוגי הציפורים שמשתמשים הוסיפו. כולם יכולים לראות את המסך, אך רק צפרים ומנהלים יכולים לערוך אותו:

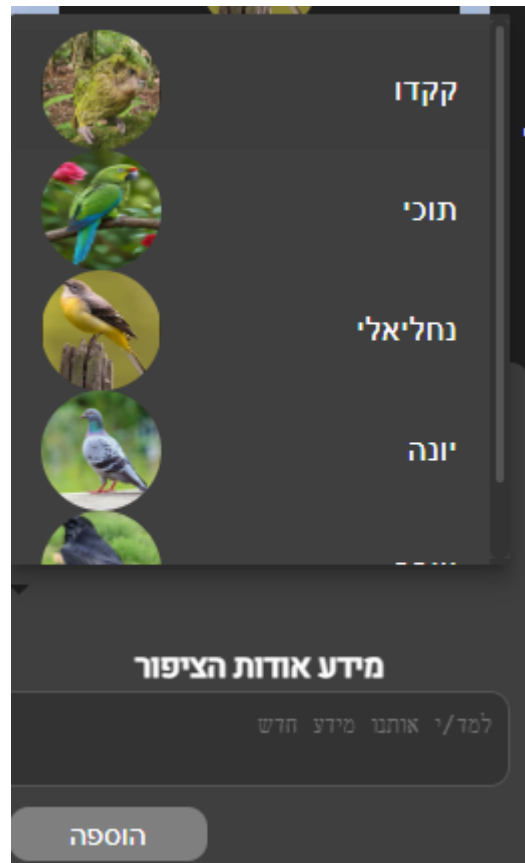


הכפתור "להוסיף מידע על ציפור" נראה כאן כי התחבר צפר. לשאר המשתמשים לא מופיע כפתור זה.

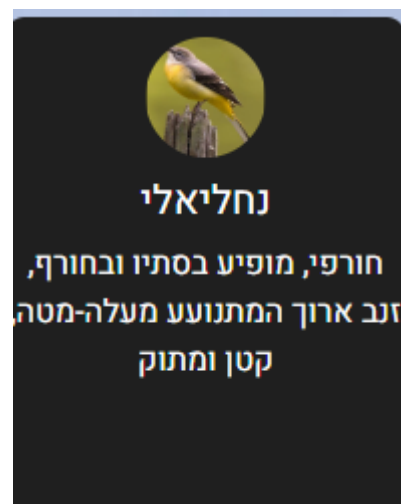
נביא דוגמה להוספת מידע על ציפור:

בעת לחיצה על הכפתור ייפתח הטופס הבא:

בחירת ציפור בדומה לבחירה בעת דיווח:



לאחר כתיבת מלל בתיבת המידע ולחיצה על "הוספה" המידע יתווסף לכרטיס המידע לכל המשתמשים.



(קטעי המידע מופרדים בפסיקים)

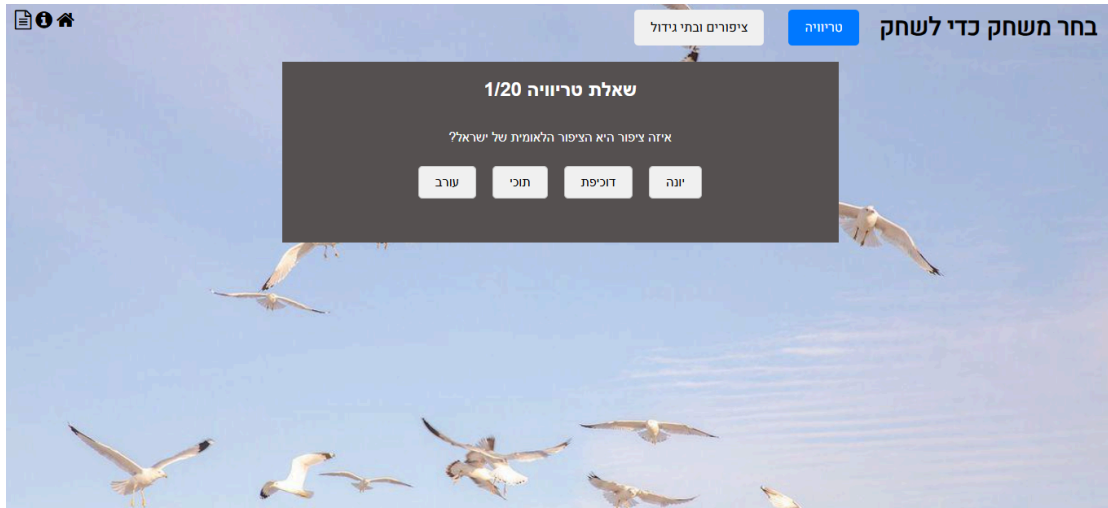
משתמש שאינו צפר או מנהל יוכל לצפות במסך המידע ולחקור על הציפורים השונות

## מסך הילדים

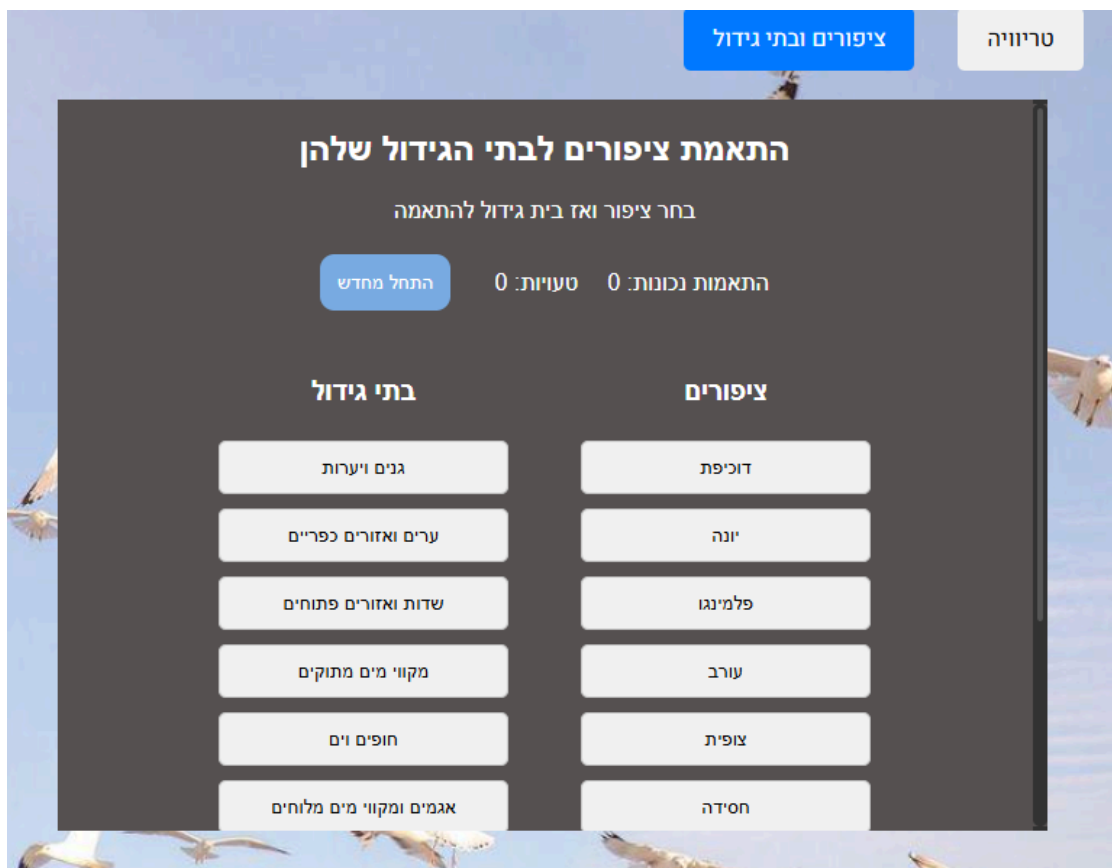
במידה ומשתמש הוא ילד, הוא יראה במסך הבית את תפריט הניווט הבא:



ניתן לראות שיש מסך מיוחד לילדים, בעת הכניסה אליו זה מה שיופיע:



באתר אנו מעדכנים את המשחקים מדי פעם על מנת ללמד את הילדים, ניתן לראות מספר משחקים חינוכיים שמלמדים על ציפורים - פה יש טריוויה ובנוסף התאמת בית גידול לציפור:



## צד לקוח - המימוש

בחרנו להשתמש באנגולר. על מנת לשמור את המידע בצורה גלובלית כדי שיוכל לעבור בין כל הקומפוננטות והמסכים במערכת, השתמשנו בstore של Ngrx.

למשל, מידע על המפה נשמר בstore של מפה:

```
export interface MapState {
  isListeningToLeftClick: boolean;
  clickedPosition: Position;
}

const initialState: MapState = {
  isListeningToLeftClick: false,
  clickedPosition: { longitude: 0, latitude: 0 },
};

export const mapReducer = createReducer(
  initialState,
  on(setClickedPosition, (state: MapState, { position }) => ({
    ...state,
    position,
  })),
  on(startListeningToClickedPosition, (state: MapState) => ({
    ...state,
    isListeningToLeftClick: true,
  })),
  on(stopListeningToClickedPosition, (state: MapState) => ({
    ...state,
    isListeningToLeftClick: false,
  })),
);

export const selectMapState = createFeatureSelector<MapState>('map');
```

שומרים את המיקום שנבחר והאם כרגע המפה מאזינה ללחיצת עכבר.

לדיווחים store:

```
export type ReportsState = {
  reportsForMap: BirdReport[],
  myReports: BirdReport[],
  reportsToApprove: BirdReport[],
};

const initialState: ReportsState = {
  reportsForMap: [],
  myReports: [],
  reportsToApprove: [],
};

export const reportsReducer = createReducer(
  initialState,
  on(setMyReports, (state: ReportsState, { reports }) => {
    return { ...state, myReports: reports };
  }),
  on(setReportsToApprove, (state: ReportsState, { reports }) => {
    return { ...state, reportsToApprove: reports };
  }),
  on(setReportsForMap, (state: ReportsState, { reports }) => {
    return { ...state, reportsForMap: reports };
  }),
);
```

שומר בקשות למפה, בקשות שלי ובקשות שאני צריך לאשר (רק למשתמש שיוכל לאשר).

למשתמש - שומר את המשתמש הנוכחי ורשימת משתמשים שיכולים לאשר store

```
export type UserState = {
  errorMsg: string;
  user: User;
  approvers: string[];
};

const initialState: UserState = {
  errorMsg: '',
  user: null,
  approvers: []
};

export const userReducer: ActionReducer<UserState, Action> = createReducer(
  Show usages
  ⚡ edens
  initialState,
  on(setUser, (state: UserState, { user }) => { user: User, errorMsg: string, approvers: string[] => {
    return { ...state, user };
  }},
  on(setErrorMsg, (state: UserState, { msg }) => { msg: string } & TypedAction<UserActions...> ) => { errorMsg: string, user: User, approvers: string[] => {
    return { ...state, errorMsg: msg };
  }},
  on(setApprovers, (state: UserState, { approvers }) => { approvers: string[] } & TypedAction<UserActions...> ) => { approvers: string[], errorMsg: string, user: User => {
    return { ...state, approvers };
  }},
);
```

בכדי לבצע בקשות לצד השרת, אנחנו משתמשים ב־apollo. קוראים ל־Action של ngrx, לו מאזין effect ומשם שולחים את הבקשה לשרת ואחר כך מספקים ל־store את המידע שהגיע מהשרת.

למשל התחברות:

```
signIn$: Observable<({ token: string } & TypedAction<UserActions...> )> = createEffect(() => Observable<({ token: string } & TypedAction<UserActions...> )> =>
  this.actions$.pipe(
    ofType(signIn),
    switchMap((action: { username: string, password: string } & TypedAction<UserActions...> ) => Observable<ApolloQueryResult<unknown>> ) =>
      this.apollo.query({
        query: signInQuery(action.username, action.password),
      })
    ),
    map((res: ApolloQueryResult<unknown> ) => { token: string } & TypedAction<UserActions...> ) => setTokenInCookie({ token: res.data['login'] })),
    catchError((err: any, caught: Observable<({ token: string } & TypedAction<UserActions...> )> ) => Observable<({ msg: string } & TypedAction<UserActions...> )> => of(setErrorMsg({ msg: err.message })))
  )
);
```

והרשמה:

```
signUp$: Observable<({ token: string } & TypedAction<UserActions...> )> = createEffect(() => Observable<({ token: string } & TypedAction<UserActions...> )> =>
  this.actions$.pipe(
    ofType(signUp),
    switchMap((action: { username: string, password: string, role: string } & TypedAction<UserActions...> ) => Observable<MutationResult<unknown>> ) =>
      this.apollo.mutate({
        mutation: signUpMutation(
          action.username,
          action.password,
          action.role
        ),
      })
    ),
    map((res: MutationResult<unknown> ) => { token: string } & TypedAction<UserActions...> ) => setTokenInCookie({ token: res.data['signUp'] })),
    catchError((err: any, caught: Observable<({ token: string } & TypedAction<UserActions...> )> ) => Observable<({ msg: string } & TypedAction<UserActions...> )> => of(setErrorMsg({ msg: err.message })))
  )
);
```

## קונפיגורציה של Apollo -

```
providers: [  
  {  
    provide: APOLLO_OPTIONS,  
    useFactory: (httpLink: HttpLink) => { link: ApolloLink, cache: InMemoryCache... => {  
      const authLink = new ApolloLink((operation: Operation, forward: NextLink) => {  
        const tokenWithBearer = 'Bearer ' + localStorage.getItem('token');  
        operation.setContext({  
          headers: {  
            Authorization: tokenWithBearer,  
            'x-apollo-operation-name': operation.operationName,  
          },  
        });  
        return forward(operation);  
      });  
  
      const link = ApolloLink.from([  
        authLink,  
        httpLink.create({ uri: process.env['SERVER_URI'] })  
      ]);  
  
      return {  
        link,  
        cache: new InMemoryCache(),  
        defaultOptions: {  
          query: {  
            fetchPolicy: 'no-cache',  
          },  
        },  
      };  
    },  
  },  
];
```

כל effect של angular יכול להאזין גם להרשאות שהוזנו בstore (ע"פ הrolen של המשתמש):

```
getAllReportsToApprove$: Observable<reports: BirdReport[] > & Ty... = createEffect(() => Observable<reports: BirdReport[] > & Ty... =>  
  this.actions$.pipe(  
    ofType(fetchReportsToApprove),  
    withLatestFrom(this.store.select(selectIsAuthorizedToApprove)),  
    filter(([, isAuthorizedToApprove]) => isAuthorizedToApprove),  
    switchMap(() =>  
      this.apollo.query({  
        query: fetchReportsToApproveQuery,  
      })  
    ),  
    map((res) =>  
      setReportsToApprove({ reports: res.data['getReportsToApprove'] })  
    )  
  )  
);
```

למשל, למשתמש שלא יכול לאשר בקשות אין סיבה לשלוח את הבקשות לאישור, לכן מפלטרים את הeffect לפי ההרשאה המתאימה. באופן כללי, כל פעולה במערכת מסוננת לפי הרשאה מתאימה.

דוגמה לmutation:

```
addReport$ = createEffect(() =>
  this.actions$.pipe(
    ofType(addReport),
    switchMap(({ report }) =>
      this.apollo.mutate({
        mutation: addReportMutation(report),
      })
    ),
    switchMap(() => [
      fetchMyReports(),
      fetchReportsForMap(),
      stopListeningToClickedPosition(),
    ])
  )
)
```

זוהי המוטציה של הוספת דיווח. נשים לב שכאן כולם יכולים להוסיף, אבל במידה והמשתמש צפר או מנהל (המידע מועבר בheaders), הבקשה תאושר אוטומטית אחרת היא תהיה ממתינה. לאחר העדכון נבצע שליפה חזרה של כל הדברים שצריך לעדכן.

אישור בקשה - רק עם הרשאה מתאימה:

```
approveReport$ = createEffect(() =>
  this.actions$.pipe(
    ofType(approveReport),
    withLatestFrom(this.store.select(selectIsAuthorizedToApprove)),
    filter(([action, isAuthorized]) => isAuthorized),
    switchMap(({ reportId }) =>
      this.apollo
        .mutate({
          mutation: approveReportMutation(reportId),
        })
        .pipe(
          tap(() => {
            Swal.fire({
              title: 'הבקשה אושרה בהצלחה',
              icon: 'success',
            });
          }),
          map(() => fetchReportsToApprove())
        )
    )
  )
);
```

## ניהול הרשאות - צפייה במידע

לאחר שהמידע מגיע לstore של המשתמש, נוצרים selectors-ים הבאים, מהם אנחנו יכולים לשלוף את המידע הרלוונטי בנוגע להרשאות:

```
export const selectRole : MemoizedSelector<object, UserRoleEnum, (s... = createSelector( Show usages  ⚙️ edens
  selectUser,
  (user: User) : UserRoleEnum => user?.role
);

export const selectIsChild : MemoizedSelector<object, boolean, (s1 Us... = createSelector( Show usages  ⚙️ edens
  selectRole,
  (role: UserRoleEnum) : boolean => role === UserRoleEnum.CHILD
);

export const selectIsAuthorizedToApprove : MemoizedSelector<object, boolean, (s1 Us... = createSelector( Show usages  ⚙️ edens
  selectRole,
  (role: UserRoleEnum) : boolean => role === UserRoleEnum.ADMIN || role === UserRoleEnum.BIRD_WATCHER
);

export const selectApprovers : MemoizedSelector<object, string[], (s1 U... = createSelector( Show usages  ⚙️ edens
  selectUserState,
  (state: UserState) : string[] => state.approvers
)
```

במסך הניווט (navbar), ניתן לראות תנאי לכפתור עמוד הילדים - רק אם אתה ילד:

```
<i class="fa fa-sign-out" (click)="logout()" title="יציאה"></i>
<i class="fa fa-info-circle" (click)="toRoute('info')" title="לעמוד המידע"></i>
<i class="fa fa-child" *ngIf="isChild$ | async" (click)="toRoute('children')" title="לעמוד הילדים"></i>
<i class="fa fa-file-text-o" (click)="toRoute('reports')" title="לעמוד הבקשות"></i>
```

בעמוד הבקשות, אם המשתמש רשאי לאשר בקשות יופיעו לו 2 עמודים (ואם לא - עמוד אחד)

ניתן לראות זאת כאן:

```
<div class="username">
  שם,
  {{currentUsername$ | async}}
</div>
<ul class="tabs" *ngIf="isAuthorized$ | async">
  <li (class.active)="selectedTab === 'my-reports'" (click)="selectTab('my-reports')">הבקשות שלי</li>
  <li (class.active)="selectedTab === 'reports-to-approve'" (click)="selectTab('reports-to-approve')">בקשות מחתינות לאישור</li>
</ul>
```

הקומפוננטה של בקשות לאישור תופיע רק אם יש בקשה מתאימה:

```
<app-reports-to-approve *ngIf="(isAuthorized$ | async) && selectedTab === 'reports-to-approve'"
  [reports]="(reportsToApprove$ | async)"
  [currentUser]="currentUsername$ | async"></app-reports-to-approve>
```

באופן דומה כך כל הקומפוננטות מתמודדות עם הרשאות במערכת.

## קומפוננטות במערכת

גם בצד הלקוח יש לנו auth guard שאחראי על לבדוק אם המשתמש מחובר ואם כן לנתב אותו למסך הבית, אחרת יוחזר למסך ההתחברות/הרשמה.

```
@Injectable({ Show usages: ± edens
  providedIn: 'root',
})
export class AuthGuard implements CanActivate {
  constructor(private router: Router) {} no usages ± edens

  canActivate(): boolean { no usages ± edens
    const token: string = localStorage.getItem('token');
    if (token) {
      const user: JwtPayload = decode(token);

      return !!user['role'];
    } else {
      this.router.navigate(['login']);
      return false;
    }
  }
}
```

הguard בודק אם יש טוקן קיים. אם כן, הוא בודק את הpayload של הtoken ובודק אם יש לו role, אם אין לו, ננתב אותו בחזרה לעמוד ההתחברות. בעליית מערכת שולחים את הבקשה verifyToken שבודקת אם יש token ואם כן מוודאת שהוא של המערכת שלנו בצד השרת. כך נוכל לדעת שבבדיקה של הguard הtoken נוצר אצלו במערכת ולא הוזן ידנית.

הrouter של angular יודע להעביר את הנתיב לקומפוננטה לפי המערך הבא שהגדרנו לו:

```

export const appRoutes: Route[] = [
  {
    path: '',
    component: HomeComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'login',
    component: LoginPageComponent
  },
  {
    path: 'children',
    component: ChildrenPageComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'reports',
    component: ReportsPageComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'info',
    component: BirdInfoPageComponent,
    canActivate: [AuthGuard]
  }
];

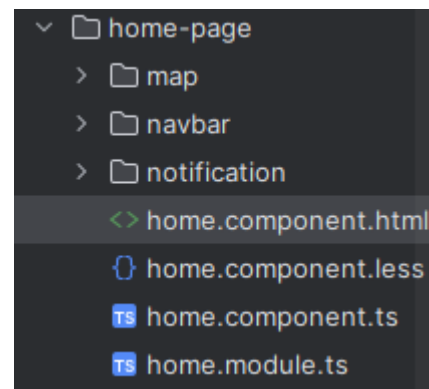
```

- ניתן לראות שכל הקומפוננטות מלבד מסך ההתחברות דורשות שימוש בGuard כדי לוודא שהמשתמש אכן מחובר. בעת ניתוח תיפתח הקומפוננטה המתאימה וקישור המערכת יתעדכן בהתאם.

**קומפוננטת מסך הבית** - השתמשנו בחלוקה נכונה של קומפוננטות כדי להפריד תלויות ולכן מסך הבית ייראה כך: (מכיל בתוכו קומפוננטת מפה וקומפוננטת navbar). באופן זה אנו שומרים על קוד קריא ולא מעמיסים על קובץ html בודד.

```
<div class="home">
  <app-navbar></app-navbar>
  <app-map></app-map>
</div>
```

סידור הקבצים הוא כזה:



לכל קומפוננטה קובץ html - מבנה, less - עיצוב, ts - לוגיקה.  
**קומפוננטת המפה:** משתמשת ב-Hooks מיוחדים של אנגולר

- ngAfterViewInit

הפונקציה תיקרא אחרי שהhtml נטען.

```
@Component({
  selector: 'app-map',
  templateUrl: 'main-map.component.html',
  styleUrls: ['main-map.component.less'],
})
export class MainMapComponent implements AfterViewInit {
  isListening$: Observable<boolean> = this.store.select(selectIsListeningToClickedPosition);
  constructor(private mapService: MapService, private store: Store) {}

  ngAfterViewInit(): void {
    this.mapService.initializeMap('cesiumContainer');
    this.store.dispatch(drawBirds());
  }
}
```

בפונקציה נאתחל את המפה בעזרת service ייחודי, ונקרא לפעולת ציור הציפורים על המפה.

## קומפוננט navbar - לוגיקה:

```
@Component({ Show usages  ⚡ edens
  selector: 'app-navbar',
  templateUrl: 'navbar.component.html',
  styleUrls: ['navbar.component.less'],
})
export class NavbarComponent {
  username$ : Observable<string> = this.store.select(selectUsername);
  isChild$ : Observable<boolean> = this.store.select(selectIsChild);
  myReportsOnMap$ : Observable<BirdReport[]> = this.store.select(selectMyReportsOnMap);
  islistening : boolean = false;

  constructor( no usages  ⚡ edens
    private store: Store,
    private router: Router
  ) {}

  logout() : void { no usages  ⚡ edens
    this.store.dispatch(logOut());
  }

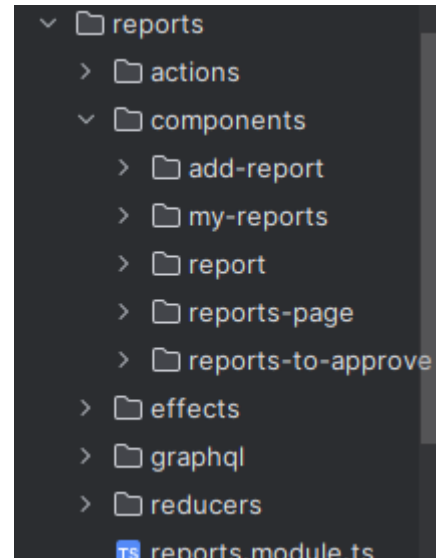
  toRoute(route: appRoutesType) : void { no usages  ⚡ edens
    this.router.navigate([route])
  }

  togglelistening() : void { no usages  ⚡ edens
    this.islistening ?
      this.store.dispatch(stopListeningToClickedPosition())
      : this.store.dispatch(startListeningToClickedPosition());
    this.islistening = !this.islistening;
  }
}
```

פה מנוהלת לוגיקת התפריט - מעבר לעמודים השונים, הוספת דיווח על המפה (התחלת האזנה ללחיצה על המפה), מוצגים הדיווחים שלי על המפה ועוד. כל המידע הנ"ל מופיע כאן.

**קומפוננט מסך הבקשות** - תכיל את כל הקומפוננטות הרלוונטיות למסך זה, הבקשות שלי, בקשות לאשר (אם אני צפר או מנהל), בקשה (קומפוננטה גנרית), והוספת בקשה (אמנם זה מופיע במפה אך מבחינה לוגית הקומפוננטה שייכת לstore הזה ולכן תופיע כאן).

אנו משתמשים בשיטת ההפרדה לקומפוננטה "חכמה" וקומפוננטה "טיפשה". כלומר, ישנה קומפוננטה ראשית שתפקידה לנהל את המידע, לעשות את הפעולות הדרושות עליו, בעוד תפקידה של הקומפוננטה הטיפשה הינו לקבל אותו ולהציגו בלבד.



קומפוננטת המסך (reports page) תכיל את כל המידע (לוקחת אותו מהstore) ותעביר אותו לקומפוננטות השונות - קומפוננטה "חכמה":

```
export class ReportsPageComponent {
  myReports$: Observable<BirdReport[]> = this.store.select(selectMyReports);
  currentUsername$: Observable<string> = this.store.select(selectUsername);
  approvers$: Observable<string[]> = this.store.select(selectApprovers);
  isAuthorized$: Observable<boolean> = this.store.select(selectIsAuthorizedToApprove);
  reportsToApprove$: Observable<BirdReport[]> = this.store.select(selectReportsToApprove);
  isChild$: Observable<boolean> = this.store.select(selectIsChild);
  selectedTab: 'my-reports' | 'reports-to-approve' = 'my-reports';
  constructor(private store: Store, private router: Router) {}

  selectTab(tab: 'my-reports' | 'reports-to-approve') {
    this.selectedTab = tab;
  }

  toRoute(route: appRoutesType) {
    this.router.navigate([route]);
  }
}
```

דרך לשמור על גנריות במערכת היא לייצר קומפוננט report שתהיה כללית לכל המערכת, והיא נראית כך:

```
<div class="report">
  <div class="header">
    <div class="title">
      דיווח על - {{report.birdName}}
      {{report.reportingUser === currentUser ? 'ידיד' : 'ידיד' + report.reportingUser}}
    </div>
    <img [src]="imageUrl">
  </div>
  <div class="body">
    <div class="nz" *ngIf="report.town">- דיווח מאזור {{report.town}}</div>
    <div class="date">
      דיווח בתאריך:
      {{report.reportDate | date:'dd/MM/yyyy'}}
      בשעה:
      {{report.reportDate | date:'hh:mm'}}
    </div>
    <div class="footer">
      <div class="status-dot" [style.background-color]="statusToColor[report.status]"></div>
      <span>{{report.status | statusToText : report.approvingUser: currentUser}}</span>
      <div *ngIf="report.status === 'PENDING' && !report.approvingUser && currentUser === report.reportingUser">
        <mat-select (valueChange)="approverChanged($event)" class="custom-select">
          <mat-option *ngFor="let approver of approvers" [value]="approver">
            {{approver}}
          </mat-option>
        </mat-select>
      </div>
      <div class="actions" *ngIf="currentUser !== report.reportingUser">
        <i (click)="reject()" class="fa fa-times" title="דחייה"></i>
        <i (click)="approve()" class="fa fa-check" title="אשר"></i>
      </div>
    </div>
  </div>
</div>
```

```
export class ReportComponent implements AfterContentInit {
  @Input() report: BirdReport;
  @Input() currentUser: string;
  @Input() approvers: string[];
  imageUrl: string;
  statusToColor: Record<ReportStatusEnum, string> = {
    APPROVED: 'green',
    REJECTED: 'red',
    PENDING: '#79ade5',
  };

  constructor(private http: HttpClient, private store: Store) {}

  ngAfterContentInit(): void {
    this.imageUrl =
      '../../../assets/birds/' + birdNameToPhoto[this.report.birdName];
  }
}
```

זוהי קומפוננטה "טיפשה", כלומר קומפוננטה שלא עושה שום לוגיקה, אלא רק מקבלת מידע (במקרה הזה בקשה) ומציגה אותו. היא פשוט כרטיס של בקשה, שאפשר להציג אותו כמה פעמים שנרצה.

במערכת השתמשנו בכמה קומפוננטות כאלה, זה עוזר להפריד תלויות ולדעת במה כל קומפוננטה נוגעת. המידע מתקבל ב, ובמידת הצורך יוצא output.

הקומפוננטות השונות משתמשות בה כמה פעמים שהן צריכות, ומזינות אליה מידע בהתאם, למשל הבקשות שלי:

```
<div class="my-reports">
  <div class="title">
    הבקשות שלי
  </div>
  <div class="reports">
    <app-report
      *ngFor="let report of myReports"
      [report]="report"
      [approvers]="approvers"
      [currentUser]="currentUser"></app-report>
    </div>
  </div>
```

ובקשות שיש לאחר:

```
<div class="reports-to-approve">
  <div class="title">
    בקשות מחתימה לאישור
  </div>
  <div class="reports">
    <app-report
      *ngFor="let report of reports"
      [report]="report"
      [currentUser]="currentUser"></app-report>
    </div>
  </div>
```

הקומפוננטה יודעת להבדיל בין סוג הבקשה (בקשה שלי או בקשה לאישור) - לפי המשתמש הנוכחי שמועבר לה, והמשתמש המאשר שמוזן בבקשה. אם הם אותו אחד, זה בקשה שאני צריך לאשר, אם לא, זה בקשה שלי.

## קומפוננטת הוספת דיווח - AddReportComponent

```
<div class="add-report">
  <div class="header">
    הוספת דיווח על ציפור
    <i class="fa fa-times" title="סגור" (click)="close()"></i>
  </div>
  <div class="body">
    <div class="bird">
      איזה ציפור ראינו?
      <app-bird-picker (birdSelected)="birdSelectionChanged($event)"></app-bird-picker>
    </div>
  </div>
  <div class="footer" (click)="add()">
    דווח
  </div>
</div>
```

## מדובר בפופאפ, שיקפוז על המפה בעזרת mat-dialog

```
@Component({ Show usages: false, *
  selector: 'app-add-report',
  templateUrl: 'add-report.component.html',
  styleUrls: ['add-report.component.less'],
})
export class AddReportComponent {
  position: Position;
  bird: string;
  constructor(@Inject(MAT_DIALOG_DATA) public data: {position: Position }, no usages: new *
    private dialogRef: MatDialogRef<AddReportComponent>,
    private store: Store,
  ) {
    this.position = this.data.position
  }

  birdSelectionChanged(bird: string) void { no usages: new *
    this.bird = bird;
  }

  add() void { no usages: new *
    const report: BirdReportInput = {birdName: this.bird, longitude: this.position.longitude, latitude: this.position.latitude}
    this.store.dispatch(addReport({report}));
    this.close();
  }

  close() void { Show usages: new *
    this.dialogRef.close();
  }
}
```

## קומפוננט מסך המידע -

תכיל את המסך הראשי, את כל כרטיסי המידע כפי שמגיעים מהשרת, ואת כפתור הוספת המידע (רק לצפר או מנהל) - זוהי קומפוננטה "חכמה":

```
export class BirdInfoPageComponent {
  birdInfos$: Observable<BirdInfo[]> = this.store.select(selectBirdInfos);
  isAuthorized$: Observable<boolean> = this.store.select(selectIsAuthorizedToApprove);
  isFormOpen: boolean = false
  birdForm: FormGroup;
  constructor(private store: Store, private formBuilder: FormBuilder) {
    no usages  ⚡ katerina
    this.birdForm = this.formBuilder.group({
      birdName: ['', Validators.required],
      description: ['', Validators.required],
    });
  }

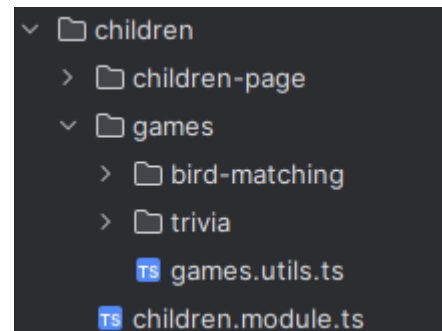
  addBirdInfo(): void {
    no usages  ⚡ katerina
    if (this.birdForm.valid) {
      this.store.dispatch(addBirdInfo({
        birdInfo: {
          birdName: this.birdForm.get('birdName').value,
          description: this.birdForm.get('description').value
        }
      })))
      this.isFormOpen = false
      this.birdForm.reset();
    }
  }

  openForm(): void {
    no usages  ⚡ katerina
    this.isFormOpen = true;
  }

  closeForm(): void {
    no usages  ⚡ katerina
    this.isFormOpen = false;
  }
}
```

```
<app-bird-info-group
  [birdInfos]="(birdInfos$ | async)">
</app-bird-info-group>
<div class="add-info" *ngIf="(isAuthorized$ | async)" (click)="openForm()">
  להוסיף מידע על ציפור
  <i class="fa fa-plus"></i>
</div>
```

קומפוננטת מסך הילדים - תכיל את כל משחקי הילדים השונים, רק ילדים יראו אותה.



```
@Component({ Show usages  # edens
  selector: 'app-children',
  templateUrl: 'children-page.component.html',
  styleUrls: ['children-page.component.less'],
})
export class ChildrenPageComponent {
  selectedGame: Game = 'trivia';

  constructor(private router: Router) {}

  selectGame(game: Game) : void { no usages
    this.selectedGame = game;
  }

  toRoute(route: appRoutesType) : void {
    this.router.navigate([route]);
  }
}
```

ניתן להוסיף סוגי משחקים באופן גנרי לסוג "Game".

## קומפוננטות המשחקים:

### משחק התאמת ציפור לבית גידול -

```
@Component({ Show usages ⚡ edens *
  selector: 'app-bird-matching-game',
  templateUrl: './bird-matching.component.html',
  styleUrls: ['./bird-matching.component.less'],
})
export class BirdMatchingGameComponent implements OnInit {
  birds: string[] = [];
  habitats: string[] = [];
  selectedBird: string | null = null;
  matches: { bird: string; habitat: string }[] = [];
  incorrectMatches: { bird: string; habitat: string }[] = [];
  currentWrongHabitat: string = null;
  currentWrongBird: string = null;

  ngOnInit() :void { Show usages ⚡ edens
    this.birds = BIRD_HABITATS.map((item : { bird: string; habitat: string } ) :string => item.bird);
    this.habitats = this.shuffle(BIRD_HABITATS.map((item : { bird: string; habitat: string } ) :string => item.habitat));
  }

  shuffle(array: string[]) :string[] { Show usages ⚡ edens
    return array.sort(() :number => Math.random() - 0.5);
  }

  selectBird(bird: string) :void { no usages ⚡ edens *
    if (this.selectedBird === bird) {
      this.selectedBird = null;
    } else {
      this.selectedBird = bird;
    }
  }
}
```

```
matchHabitat(habitat: string) :void { no usages ⚡ edens
  if (this.selectedBird) {
    const correctPair : { bird: string; habitat: string } = BIRD_HABITATS.find(
      (item : { bird: string; habitat: string } ) :boolean => item.bird === this.selectedBird && item.habitat === habitat
    );

    if (correctPair) {
      this.matches.push({ bird: this.selectedBird, habitat });
      this.birds = this.birds.filter((b :string ) :boolean => b !== this.selectedBird);
      this.habitats = this.habitats.filter((h :string ) :boolean => h !== habitat);
    } else {
      this.incorrectMatches.push({ bird: this.selectedBird, habitat });
      this.currentWrongHabitat = habitat;
      this.currentWrongBird = this.selectedBird;
      setTimeout(() :void => {
        this.currentWrongHabitat = null;
        this.currentWrongBird = null;
      }, 1000);
    }

    this.selectedBird = null;
  }
}

restartGame() :void { no usages ⚡ edens
  this.ngOnInit();
  this.matches = [];
  this.incorrectMatches = [];
}
```

ניהול המשחק מתבצע בקומפוננטה זו. המידע יישמר במשתנה גלובלי BIRD\_HABITATS שלא משתנה לרוב, הוא יישמר בקוד בקובץ ייעודי בשם bird-games-data.ts

אין צורך לשמור מידע זה במסד הנתונים, משום שזה מידע שנגיש רק למפתחים אשר ביכולתם לשנות את הערכים - דבר שלא קורה באופן תכוף, לכן אין צורך להעמיס על מסד הנתונים.

## משחק טריוויה

```
export class BirdTriviaGameComponent {
  questions : { question: string; options: string[]; an... = BIRD_TRIVIA_QUESTIONS;
  currentQuestionIndex : number = 0;
  score : number = 0;
  showResult : boolean = false;
  currentRightBird: string = null;
  currentWrongBird: string = null;

  selectAnswer(option: string) : void { no usages 1 edens
    if (option === this.questions[this.currentQuestionIndex].answer) {
      this.score++;
      this.currentRightBird = option;
    } else {
      this.currentWrongBird = option;
    }
  }

  setTimeout(() : void => {
    if (this.currentQuestionIndex < this.questions.length - 1) {
      this.currentQuestionIndex++;
    } else {
      this.showResult = true;
    }
    this.currentRightBird = null;
    this.currentWrongBird = null;
  }, 500);
}

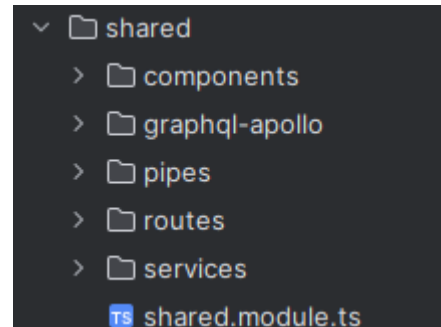
restartGame() : void { no usages 1 edens
  this.currentQuestionIndex = 0;
  this.score = 0;
  this.showResult = false;
}
}
```

## קומפוננטות גנריות נוספות

### AppBirdPicker -

קומפוננטת ui - לבחירת ציפור, תציג דרופאון לבחירת ציפור עם תמונות. משתמשים בה במסך הילדים, בהוספת דיווח על מפה, בהוספת מידע על ציפור. בעצם בכל מקום שיש לבחור בו ציפור.

היא שייכת לתיקיית shared - תיקייה של דברים שימושיים וחיוניים לכל המערכת



ונראית כך:

```
<div *ngIf="(birds$ | async) as birds">
  <div class="bird-picker">
    <mat-select (valueChange)="birdSelected.emit($event)" [value]="birds[0]">
      <mat-option class="mat-option-bird-picker" *ngFor="let bird of birds" [value]="bird.name">
        <img class="bird-photo-in-picker" [src]="bird.imageUrl">
        {{ bird.name }}
      </mat-option>
    </mat-select>
  </div>
</div>
```

```
@Component({
  selector: 'app-bird-picker',
  templateUrl: 'bird-picker.component.html',
  styleUrls: ['bird-picker.component.less'],
})
export class BirdPickerComponent {
  birds$: Observable<{ imageUrl: string; id: string; }> = this.store.select(selectBirdsWithPhotos)
  @Output() birdSelected: EventEmitter<string> = new EventEmitter<string>();

  constructor(private store: Store) {}
}
```

ניתן לראות שהקומפוננטה מייחצנת output - זה בעצם event שקורה, ואז קומפוננטת האב שלה יכולה להאזין לשינויים בevent הזה. במקרה זה מדובר בבחירת ציפור.

## Pipes

כלי של אנגולר שנועד לקחת משתנה ולהפוך אותו לטקסט נוח לקריאה. למשל בקומפוננטת report צריך לפרט על סטטוס הבקשה (מאושרת, ממתינה, נדחתה) כמו גם על מי אישר או דחה, האופציות השונות נראות כך:

**תוכי - דווח על ידך**  
דווח בתאריך: 23/12/2024 בשעה: 07:23  
● לא מאושר, נדחה על ידי אופיר

**עורב - דווח על ידך**  
דווח בתאריך: 05/03/2025 בשעה: 11:27  
● מאושר, אושר על ידי אופיר

**קקדו - דווח על ידי חובב טבע**  
דווח בתאריך: 05/03/2025 בשעה: 11:27  
● ממתין לאישור על ידך ❌

**קקדו - דווח על ידך**  
דווח בתאריך: 05/03/2025 בשעה: 11:27  
● ממתין לאישור, משתמש מאשר: חתול

על מנת לא לחזור על קוד בשביל הטקסט למטה, השתמשנו בpipe שייקח את המידע של הבקשה וימיר אותו לטקסט המתאים, זה נראה כך:

```
@Pipe({name: 'statusToText'}) Show usages & edens
export class ReportStatusPipe implements PipeTransform {
  transform(status: ReportStatusEnum, approvingUser?: string, currentUser?: string): string {
    let string: string = '';
    switch (status) {
      case ReportStatusEnum.APPROVED:
        string += 'מאושר, מאשר על ידי ' + approvingUser;
        break;
      case ReportStatusEnum.PENDING:
        string += 'ממתין לאישור';
        if (approvingUser === currentUser) {
          string += ' על ידך';
        } else if (approvingUser) {
          string += ' ממשתמש מאשר: ' + approvingUser;
        }
        break;
      case ReportStatusEnum.REJECTED:
        string += 'לא מאושר, נדחה על ידי ' + approvingUser;
    }
    return string;
  }
}
```

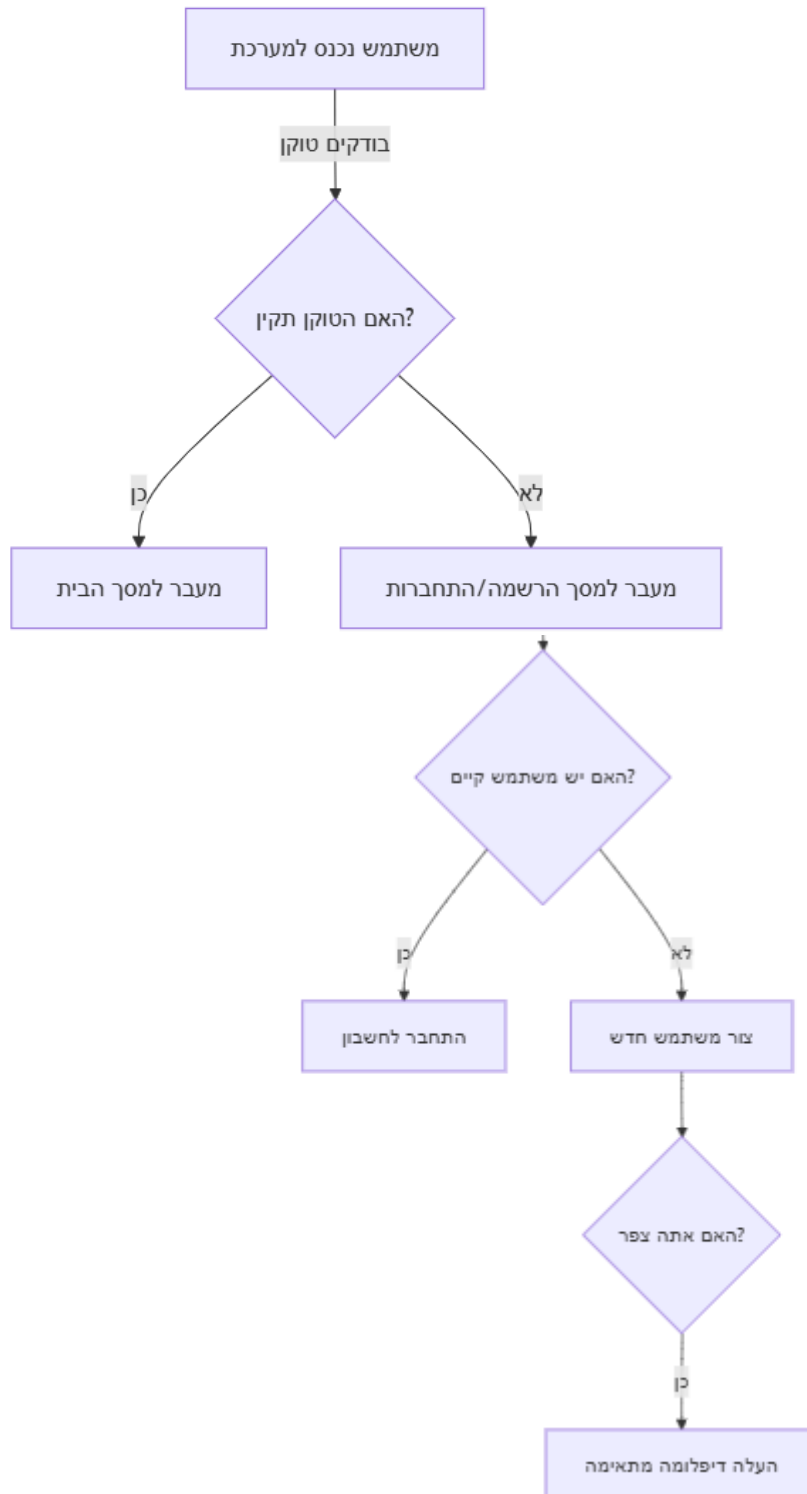
והשימוש בhtml:

```
<div class="status-report" style="background-color: #f0f0f0; padding: 10px; border: 1px solid #ccc;">
  <span>{{report.status | statusToText : report.approvingUser: currentUser}}</span>
</div>
```

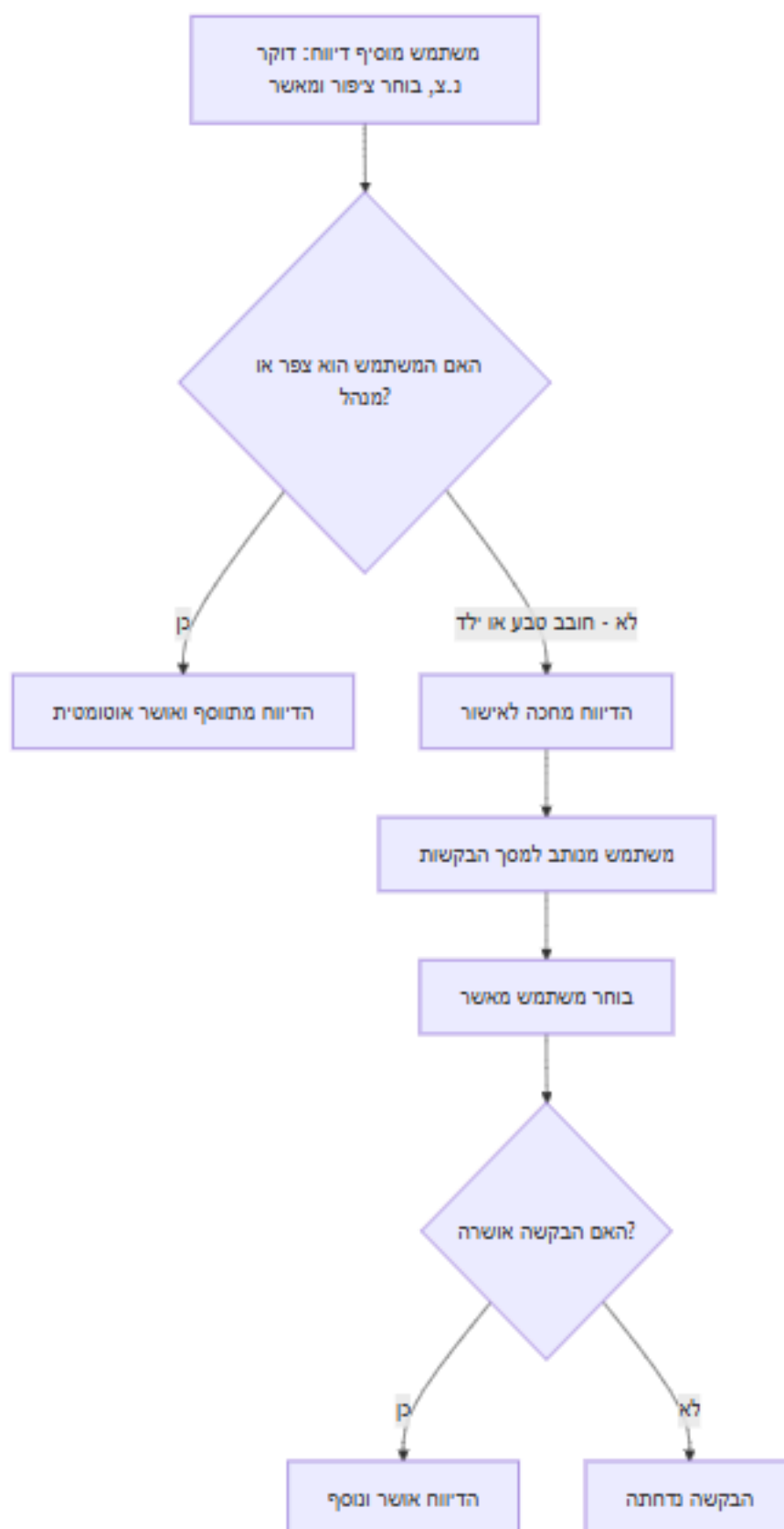
## תהליכים מרכזיים במערכת

נציג עכשיו תרשימים שמייצגים כמה תהליכים

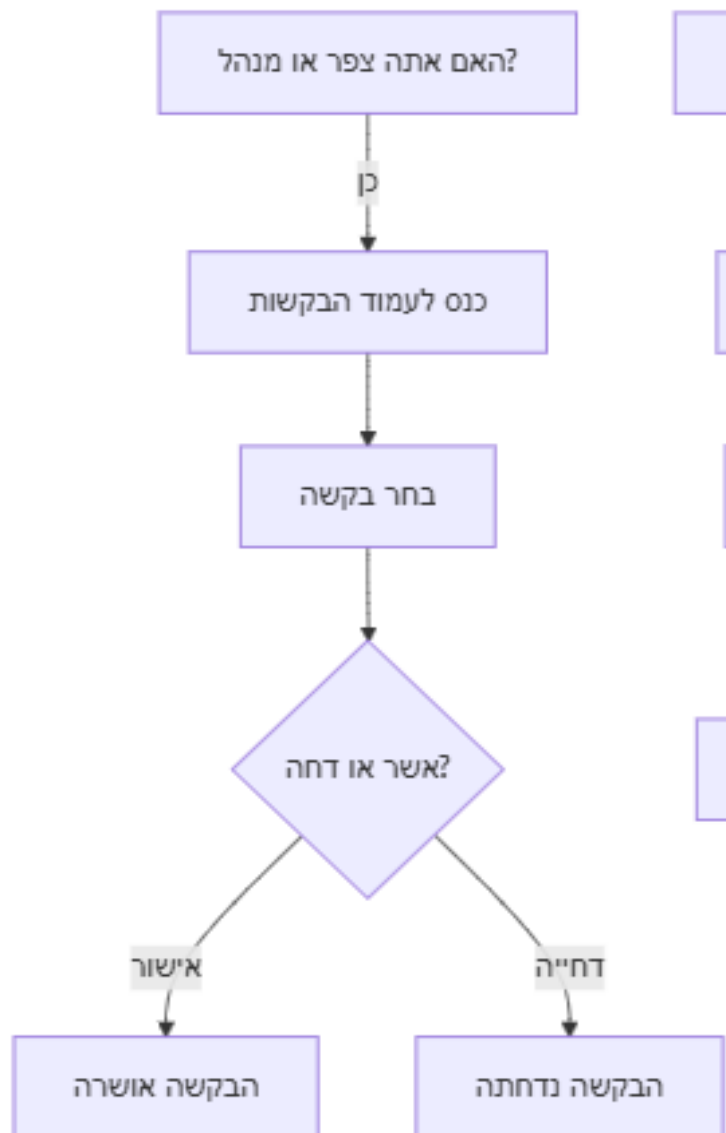
התחברות



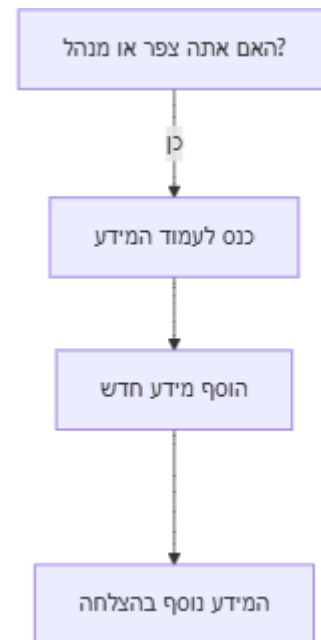
## הוספת דיווח



## אישור או דחיית בקשה



### הוספת מידע על ציפור



### הוספת ציפור - למנהלים בלבד.

## **סיכום ומסקנות**

הפרויקט פיתח את יכולתנו לקחת מערכת מבצעית ולהמיר אותה למערכת "מולבנת" תוך שמירה על הערכים המקוריים והמטרה של המערכת, על מנת להנגיש אותה גם לאנשים מחוץ לצבא.

נוסף על כך, המערכת לימדה אותנו רבות על עולם הציפורים המגוון וחיזקה את הקשר שלנו לטבע.

חשוב לציין שיש הבדל גדול בין פיתוח ברשת צהלית סגורה לבין פיתוח ברשת אזרחית. הפרויקט גרם לנו לחקור על טכנולוגיות מגוונות ועל התאמתן לפרויקט שלנו. התעמקנו בטכנולוגיות בהם אנו משתמשים ביום יום וקיבלנו המון מידע חדש עליהן, שעוזר לנו ביום יום.

התמודדנו עם אתגרים רבים במהלך הפרויקט, הקמת מסד נתונים ותכנונו ברמת ארכיטקטורה, פיתוח צד שרת ופיתוח צד לקוח לגמרי מהאפס והחיבור ביניהם, כמו גם העלאה לאוויר של המערכת.

## **ביבליוגרפיה**

[/https://nestjs.com](https://nestjs.com)  
<https://www.npmjs.com/>  
<https://sequelize.org/docs/v6/other-topics/typescript/>  
<https://jwt.io/>  
<https://www.npmjs.com/package/jwt-decode>  
<https://angular.dev/>  
<https://material.angular.io/>  
<https://the-guild.dev/graphql/apollo-angular/docs>  
<https://sweetalert2.github.io/>  
<https://ngrx.io/>  
<https://angular.dev/guide/forms>  
<https://cesium.com/>  
<https://www.pgadmin.org/>